

Ingeniería de características: desarrollo matemático e implementación en AWS

Notas complementarias al capítulo 3 de la guía MLA-C01

Documentación de AWS verificada el 24 de septiembre de 2026

Índice

1. Marco general	3
1.1. Modelo probabilístico y notación	3
1.2. Transformaciones ajustadas y fuga de información	3
1.3. Tres propiedades de invariancia	5
2. Preparación previa: valores faltantes y atípicos	7
2.1. Valores faltantes	7
2.2. Valores atípicos	9
2.2.1. Regla de la puntuación z	9
2.2.2. Regla del rango intercuartílico	10
2.2.3. Puntuación z modificada	10
2.2.4. Punto de ruptura	11
3. Características numéricas: escalamiento lineal	12
3.1. Definición y casos	12
3.2. Qué cambia y qué no cambia	12
3.3. Escala y descenso de gradiente	13
4. Características numéricas: transformaciones no lineales	14
4.1. Transformación logarítmica	14
4.2. Raíces cuadrada y cúbica	16
4.3. Transformación de Box–Cox	16
4.4. Transformación de Yeo–Johnson	17
5. Discretización	19
6. Características categóricas: codificaciones no supervisadas	20
6.1. Planteamiento	20
6.2. Codificación ordinal	21
6.3. Codificación <i>one-hot</i>	21
6.4. Codificación binaria	22
6.5. <i>Hashing</i> de características	23
7. Características categóricas: codificaciones supervisadas	25
7.1. <i>Target encoding</i>	25
7.1.1. Fuga de información dentro del entrenamiento	27
7.1.2. El orden de las medias y los árboles	28
7.2. <i>Weight of evidence</i> e <i>information value</i>	30
7.3. Resumen de codificaciones categóricas	32

8. Series de tiempo	32
8.1. Descomposición de la marca de tiempo	33
8.2. Características de rezago	33
8.3. Características de ventana móvil	34
9. Imágenes	35
9.1. Normalización de píxeles	35
9.2. Extracción clásica: convolución y gradientes	35
9.3. Vectores de características de una red preentrenada	36
9.4. Análisis de componentes principales	36
10. Texto	37
10.1. Tokenización	38
10.2. Palabras vacías	38
10.3. <i>Stemming</i> y lematización	39
10.4. Bolsa de palabras, n -gramas y TF-IDF	39
10.5. <i>Word embeddings</i>	40
11. Selección de características por filtros	42
12. Resumen: técnica y servicio	44

1. Marco general

Estas notas desarrollan matemáticamente las técnicas de preparación de datos e ingeniería de características del capítulo 3 de la guía, añaden dos codificaciones supervisadas que el capítulo no cubre (*target encoding* y *weight of evidence*) y describen, para cada técnica, el servicio de AWS con que se implementa. El nivel de formalidad es el de los textos clásicos de estadística y aprendizaje estadístico.

1.1. Modelo probabilístico y notación

Datos. Se modela cada observación como un par aleatorio (X, Y) con distribución conjunta P desconocida, donde $X = (X_1, \dots, X_p)$ es el vector de *características* (covariables) e Y la *respuesta* (variable objetivo). Cada X_j toma valores en $\mathbb{R}$ (característica numérica) o en un conjunto finito (característica categórica, más abajo). Una *muestra* es una sucesión $\mathcal{D} = ((x_1, y_1), \dots, (x_n, y_n))$ de realizaciones de n copias independientes de (X, Y) ; se escribe $x_k = (x_{k1}, \dots, x_{kp})$.

Estadísticos de una columna numérica. Una *columna* es un vector $z = (z_1, \dots, z_n) \in \mathbb{R}^n$, típicamente $z_k = x_{kj}$ para un j fijo. Se usan:

- la media $\bar{z} = n^{-1} \sum_{k=1}^n z_k$;
- las desviaciones estándar $s_n(z) = (n^{-1} \sum_k (z_k - \bar{z})^2)^{1/2}$ y $s_{n-1}(z) = ((n-1)^{-1} \sum_k (z_k - \bar{z})^2)^{1/2}$; cuando el divisor es irrelevante o se indica en el contexto se escribe s ;
- los estadísticos de orden $z_{(1)} \leq z_{(2)} \leq \dots \leq z_{(n)}$, es decir, los valores de la columna ordenados de menor a mayor;
- el *cuantil muestral* de orden $q \in [0, 1]$, definido por interpolación lineal entre estadísticos de orden: con $h = (n-1)q + 1$ y $r = \lfloor h \rfloor$,

$$Q(q) = z_{(r)} + (h - r)(z_{(r+1)} - z_{(r)}) \quad (\text{con } z_{(n+1)} := z_{(n)}).$$

Es la definición por defecto de NumPy y pandas, las bibliotecas que usa el capítulo. La mediana es $\text{med}(z) = Q(1/2)$, los cuartiles son $Q_1 = Q(1/4)$ y $Q_3 = Q(3/4)$, y el rango intercuartílico es $\text{IQR} = Q_3 - Q_1$.

Por ejemplo, para la columna de consumo de energía del capítulo,

$$z = (11, 12, 12.5, 8, 8.5, 7.5, 5, 4.7, 6, 23),$$

se tiene $n = 10$; para $q = 3/4$, $h = 7.75$, $r = 7$, y como $z_{(7)} = 11$ y $z_{(8)} = 12$, resulta $Q_3 = 11 + 0.75 \cdot 1 = 11.75$; análogamente $Q_1 = 6.375$.

Características categóricas. Una característica categórica toma valores en un conjunto finito $\mathcal{C} = \{c_1, \dots, c_K\}$ cuyos elementos se llaman *niveles*; K es la *cardinalidad*. En una muestra, $n_i = \#\{k : x_{kj} = c_i\}$ es el número de observaciones del nivel c_i .

1.2. Transformaciones ajustadas y fuga de información

Casi todas las técnicas de este documento reemplazan una característica por una función de ella cuyos parámetros se calculan con datos: la media y la desviación estándar de un escalado, el exponente λ de Box-Cox, la tabla de medias de un *target encoding*, el vocabulario de un vectorizador de texto, las direcciones de un PCA. El capítulo separa «ajustar» (**fit**) de «aplicar» (**transform**); esta sección precisa esa separación y demuestra por qué es necesaria.

Definición 1.1 (Transformación ajustada). Sean $\mathcal{X}$ el conjunto de valores posibles de X , $\mathcal{X}'$ otro conjunto y Θ un conjunto de parámetros. Una *transformación ajustada* consta de

- (I) una familia de funciones $T_\theta : \mathcal{X} \rightarrow \mathcal{X}'$, $\theta \in \Theta$, y
- (II) un *estimador* $\hat{\theta}$, es decir, una regla que a cada muestra asigna un elemento de Θ .

Dada una muestra de entrenamiento $\mathcal{D}_{\text{tr}}$, *ajustar* es calcular $\hat{\theta}(\mathcal{D}_{\text{tr}})$; *aplicar* es evaluar $T_{\hat{\theta}(\mathcal{D}_{\text{tr}})}$ en cualquier $x \in \mathcal{X}$, pertenezca o no a $\mathcal{D}_{\text{tr}}$.

Ejemplo 1.2. Estandarización de la característica j : $\Theta = \mathbb{R} \times (0, \infty)$, $T_{(a,b)}(x)$ reemplaza x_j por $(x_j - a)/b$ y deja las demás coordenadas, y $\hat{\theta}(\mathcal{D}_{\text{tr}}) = (\bar{z}, s_n(z))$ con z la columna j de $\mathcal{D}_{\text{tr}}$. Un ejemplo sin parámetros es $T(x) = \log x_j$: Θ tiene un solo elemento y ajustar no hace nada.

Para evaluar un modelo se fija una *función de pérdida* $L(y, \hat{y}) \geq 0$ (por ejemplo, $(y - \hat{y})^2$ en regresión o $\mathbf{1}\{y \neq \hat{y}\}$ en clasificación) y un *modelo ajustado* $\hat{f} : \mathcal{X}' \rightarrow \mathcal{Y}$, que también se calcula con $\mathcal{D}_{\text{tr}}$. La cantidad de interés es el *riesgo condicional*

$$R = \mathbb{E} \left[L(Y, \hat{f}(T_{\hat{\theta}}(X))) \mid \mathcal{D}_{\text{tr}} \right],$$

donde (X, Y) es una observación nueva independiente de $\mathcal{D}_{\text{tr}}$: el error esperado del procedimiento ya entrenado sobre datos futuros. Se estima con una muestra de prueba.

Proposición 1.3 (Insensatez del error de prueba). Sean $\mathcal{D}_{\text{tr}}$ y $\mathcal{D}_{\text{te}} = ((X'_1, Y'_1), \dots, (X'_m, Y'_m))$ independientes, con los pares (X'_l, Y'_l) independientes entre sí y con la distribución P de (X, Y) . Supóngase que $\hat{\theta}$ y $\hat{f}$ se calculan solo con $\mathcal{D}_{\text{tr}}$ y que $R < \infty$. Entonces $\hat{R} = m^{-1} \sum_{l=1}^m L(Y'_l, \hat{f}(T_{\hat{\theta}}(X'_l)))$ cumple $\mathbb{E}[\hat{R} \mid \mathcal{D}_{\text{tr}}] = R$.

Demostración. Sea $G(\mathcal{D}, x, y) = L(y, \hat{f}_{\mathcal{D}}(T_{\hat{\theta}(\mathcal{D})}(x)))$, donde el subíndice indica la muestra con que se calculan $\hat{f}$ y $\hat{\theta}$. Se usa la siguiente propiedad de la esperanza condicional: si U y V son independientes y $G \geq 0$, entonces $\mathbb{E}[G(U, V) \mid U] = g(U)$ con $g(u) = \mathbb{E}[G(u, V)]$. Con $U = \mathcal{D}_{\text{tr}}$ y $V = (X'_l, Y'_l)$, que son independientes por hipótesis y V tiene distribución P , se obtiene $\mathbb{E}[G(\mathcal{D}_{\text{tr}}, X'_l, Y'_l) \mid \mathcal{D}_{\text{tr}}] = g(\mathcal{D}_{\text{tr}})$, donde $g(\mathcal{D}) = \mathbb{E}[G(\mathcal{D}, X, Y)]$ con $(X, Y) \sim P$. Aplicando la misma propiedad con $V = (X, Y)$ se tiene también $R = g(\mathcal{D}_{\text{tr}})$. Promediando sobre l y usando la linealidad de la esperanza condicional, $\mathbb{E}[\hat{R} \mid \mathcal{D}_{\text{tr}}] = g(\mathcal{D}_{\text{tr}}) = R$. $\square$

La hipótesis de que $\hat{\theta}$ no use la muestra de prueba es necesaria.

Contraejemplo 1.4. Sea X real con distribución continua e $Y \in \{0, 1\}$ independiente de X con $\mathbb{P}(Y = 1) = 1/2$, de modo que todo clasificador tiene riesgo $1/2$ con pérdida $\mathbf{1}\{y \neq \hat{y}\}$. Por la continuidad, con probabilidad 1 las X'_l son distintas entre sí. Defínase un parámetro estimado con $\mathcal{D}_{\text{tr}} \cup \mathcal{D}_{\text{te}}$: $\hat{\theta}$ es la tabla $\{(X'_l, Y'_l)\}_{l=1}^m$, y $T_{\hat{\theta}}(x)$ añade a x la coordenada Y'_l si $x = X'_l$ y 0 en otro caso. El clasificador que devuelve esa coordenada añadida tiene $\hat{R} = 0$. Una observación verdaderamente nueva coincide con alguna X'_l con probabilidad 0 (nuevamente por la continuidad), así que el clasificador predice 0 y su riesgo es $\mathbb{P}(Y = 1) = 1/2$.

El contraejemplo es extremo, pero su mecanismo es el de cualquier parámetro que use las respuestas de prueba; las secciones 7.1 y 7.2 muestran que con *target encoding* aparece incluso dentro del entrenamiento. Cuando el parámetro no usa respuestas (una media, un mínimo), la proposición deja igualmente de aplicarse y la magnitud del sesgo depende del caso. La proposición justifica también la regla operativa del capítulo: calcular $\hat{\theta}$ una vez con entrenamiento y aplicar la misma $T_{\hat{\theta}}$ en validación, prueba y producción. Se llama *training-serving skew* a la situación en que la transformación aplicada en producción difiere de la $T_{\hat{\theta}}$ con que se entrenó $\hat{f}$; entonces $\hat{f}$ recibe entradas con otra distribución que la de entrenamiento, y R deja de describir su desempeño.

Implementación en AWS. Una transformación ajustada vive en uno de tres lugares. (i) **Amazon SageMaker Data Wrangler** (integrado hoy en SageMaker Canvas; la versión de Studio Classic sigue documentada): cada paso de un *data flow* que aprende de los datos (codificar categóricas, vectorizar texto, tratar atípicos o faltantes) guarda sus parámetros, calculados por defecto sobre la muestra importada (las primeras 50 000 filas); al exportar puede reajustarlos (*refit*) sobre el dataset completo, y los parámetros se trasladan al *inference pipeline*. (ii) **AWS Glue DataBrew**: una *receta* es una lista JSON de acciones que un trabajo serverless aplica al dataset en Amazon S3; los estadísticos que usan sus acciones se calculan sobre el dataset al que se aplica la receta en cada ejecución. (iii) **SageMaker Processing**: un contenedor (por ejemplo, el de scikit-learn) ejecuta un script propio; es la vía para técnicas sin paso integrado y la que da control explícito de **fit** sobre entrenamiento y **transform** sobre lo demás. Los parámetros que deben consultarse en inferencia (por ejemplo, tablas de codificación) pueden guardarse en S3 o en un *feature group* de **SageMaker Feature Store**, cuyo *online store* los sirve con baja latencia.

1.3. Tres propiedades de invariancia

La conveniencia de una transformación depende del modelo que la recibe. Esta sección demuestra qué transformaciones no alteran a los árboles de decisión y a la regresión lineal, y muestra que otros modelos sí son sensibles; los resultados se usan en todo el documento.

Definición 1.5 (Árbol de regresión construido por CART). Sea $\mathcal{D}$ una muestra con respuesta numérica. Para $A \subset \{1, \dots, n\}$ no vacío, sean $\bar{y}_A$ la media de las y_k con $k \in A$ y $SC(A) = \sum_{k \in A} (y_k - \bar{y}_A)^2$. Un *nodo* es un conjunto de índices N . Un *corte* del nodo N sobre la columna j con umbral $t \in \mathbb{R}$ es la partición de N en $N_{\leq} = \{k \in N : x_{kj} \leq t\}$ y $N_{>} = N \setminus N_{\leq}$, con ambos no vacíos. El algoritmo parte de $N = \{1, \dots, n\}$ y, mientras una regla de parada que depende solo de N y de $(y_k)_{k \in N}$ no se cumpla, elige entre todos los cortes de todas las columnas uno que maximice $\Delta = SC(N) - SC(N_{\leq}) - SC(N_{>})$, desempata con una regla que depende solo de la columna y de la partición $\{N_{\leq}, N_{>}\}$ (sin distinguir cuál de los dos conjuntos es el de la izquierda), y repite en cada hijo. Las *hojas* son los nodos donde se detiene. Para clasificación, SC se reemplaza por $|A|$ veces el índice de Gini de las respuestas en A , que es $1 - \sum_c \hat{p}_c^2$ con $\hat{p}_c$ la proporción de la clase c en A .

Las condiciones sobre la regla de parada y el desempate no son técnicas: si el desempate dependiera del valor numérico del umbral t , el resultado podría cambiar al transformar la columna aunque los cortes candidatos fueran los mismos.

Lema 1.6. Sea $g : \mathbb{R} \rightarrow \mathbb{R}$ estrictamente creciente. Para todo nodo N y toda columna j , las particiones de N obtenibles con cortes sobre $(x_{kj})_{k \in N}$ coinciden con las obtenibles con cortes sobre $(g(x_{kj}))_{k \in N}$.

Demostración. Sean $v_1 < \dots < v_r$ los valores distintos de $(x_{kj})_{k \in N}$. Para un umbral t , sea $i(t)$ el número de v_i que son $\leq t$; entonces $N_{\leq} = \{k \in N : x_{kj} \leq v_{i(t)}\}$ (vacío si $i(t) = 0$), de modo que las particiones obtenibles son las $\{k : x_{kj} \leq v_i\}$, $i = 1, \dots, r-1$. Como g es estrictamente creciente, $x_{kj} \leq v_i$ equivale a $g(x_{kj}) \leq g(v_i)$, y los valores distintos de la columna transformada son $g(v_1) < \dots < g(v_r)$. Aplicando el mismo razonamiento a la columna transformada, sus particiones obtenibles son las $\{k : g(x_{kj}) \leq g(v_i)\} = \{k : x_{kj} \leq v_i\}$, $i = 1, \dots, r-1$: las mismas. $\square$

Proposición 1.7 (Árboles e invariancia monótona). Si se reemplaza la columna j por $(g(x_{kj}))_k$ con g estrictamente creciente, el árbol de la Definición 1.5 produce las mismas hojas (como conjuntos de índices).

Demostración. Por inducción sobre los nodos en el orden en que el algoritmo los crea. La raíz es la misma. Si un nodo N es el mismo en ambas ejecuciones, la regla de parada toma la misma decisión porque solo depende de N y de las respuestas. Por el Lema 1.6 (aplicado a la columna j ; las demás columnas no cambian), el conjunto de pares (columna, partición) candidatos es el mismo; el criterio Δ depende solo de la partición y de las respuestas, y el desempate solo de la columna y la partición. Por tanto se elige la misma partición y los hijos coinciden. $\square$

Los umbrales numéricos cambian (en la columna transformada el umbral aplicable es $g(v_i)$), pero ninguna observación cambia de hoja. El mismo argumento vale para los métodos que agregan árboles de este tipo (bosques aleatorios, *gradient boosting*), siempre que sus cortes candidatos se definan como arriba; las implementaciones que discretizan primero las columnas en histogramas de ancho fijo no cumplen la hipótesis.

Definición 1.8 (Mínimos cuadrados con intercepto). Sea $X \in \mathbb{R}^{n \times p}$ y $Z = [\mathbf{1} \ X]$, con $\mathbf{1} \in \mathbb{R}^n$ el vector de unos. Un estimador de mínimos cuadrados es cualquier $\hat{\beta}$ que minimice $\|y - Z\beta\|^2$; el vector de valores ajustados es $\hat{y} = Z\hat{\beta}$.

Lema 1.9. $\hat{y}$ no depende del minimizador elegido: es la proyección ortogonal de y sobre $\text{col}(Z)$, el espacio generado por las columnas de Z .

Demostración. Sea π la proyección ortogonal de y sobre $\text{col}(Z)$, caracterizada por $\pi \in \text{col}(Z)$ e $y - \pi \perp \text{col}(Z)$. Para todo β , por el teorema de Pitágoras, $\|y - Z\beta\|^2 = \|y - \pi\|^2 + \|\pi - Z\beta\|^2$, porque $\pi - Z\beta \in \text{col}(Z)$. El mínimo es $\|y - \pi\|^2$ y se alcanza exactamente cuando $Z\beta = \pi$. $\square$

Proposición 1.10 (Mínimos cuadrados e invariancia afín). Sea $\tilde{X} = XD + \mathbf{1}c^\top$ con D diagonal con entradas no nulas y $c \in \mathbb{R}^p$, es decir, cada columna j se reemplaza por $d_j x_{\cdot j} + c_j$ con $d_j \neq 0$. Los valores ajustados por mínimos cuadrados con intercepto son los mismos con X y con $\tilde{X}$.

Demostración. $[\mathbf{1} \ \tilde{X}] = [\mathbf{1} \ X]M$ con $M = \begin{pmatrix} 1 & c^\top \\ 0 & D \end{pmatrix}$, que es invertible porque $\det M = \det D \neq 0$. Si $u = [\mathbf{1} \ \tilde{X}]\alpha$, entonces $u = [\mathbf{1} \ X](M\alpha)$, y si $u = [\mathbf{1} \ X]\beta$, entonces $u = [\mathbf{1} \ \tilde{X}](M^{-1}\beta)$; los espacios columna coinciden. Por el Lema 1.9, los valores ajustados coinciden. $\square$

La hipótesis de intercepto es necesaria para la parte de traslación: sin él, $\text{col}(X)$ y $\text{col}(X + \mathbf{1}c^\top)$ pueden diferir (con $p = 1$, $x = (1, 2)^\top$ genera la recta de dirección $(1, 2)$ y $x + 1 = (2, 3)^\top$ otra distinta).

Contraejemplo 1.11 (Regularización y vecinos más cercanos). (a) Regresión *ridge* con una característica y sin intercepto: el estimador que minimiza $\sum_k (y_k - \beta x_k)^2 + \lambda \beta^2$ ($\lambda > 0$) es $\hat{\beta} = \sum_k x_k y_k / (\sum_k x_k^2 + \lambda)$, que se obtiene igualando a cero la derivada de esa función cuadrática estrictamente convexa; el valor ajustado en un punto $u \in \mathbb{R}$ es $\hat{\beta}u$. Si la característica se reemplaza por ax , $a \neq 0$ (en entrenamiento y en el punto, que pasa a ser au), el valor ajustado pasa a ser $au \cdot \frac{a \sum_k x_k y_k}{a^2 \sum_k x_k^2 + \lambda} = u \frac{\sum_k x_k y_k}{\sum_k x_k^2 + \lambda/a^2}$, que depende de a : cambiar la unidad de la característica equivale a cambiar la penalización. (b) El clasificador del vecino más cercano asigna a un punto q la respuesta del punto de entrenamiento más cercano en distancia euclídea. Con $q = (0, 0)$, $B = (1, 0)$ y $C = (0, 2)$, el más cercano es B (distancias 1 y 2). Si la primera coordenada se multiplica por 10, $B = (10, 0)$ y el más cercano pasa a ser C .

Los árboles no necesitan escalado (Proposición 1.7), la regresión lineal sin penalizar tampoco para predecir (Proposición 1.10), y los métodos penalizados y basados en distancias sí (Contraejemplo 1.11); la Proposición 3.7 añade el descenso de gradiente a esta última lista.

2. Preparación previa: valores faltantes y atípicos

2.1. Valores faltantes

Se considera una característica numérica X que puede faltar y un vector W de otras características siempre observadas. Sea $R = 1$ si X falta y $R = 0$ si se observa; en la muestra, r_k es el valor de R en la observación k y $O = \{k : r_k = 0\}$ el conjunto de observaciones con X presente, de tamaño n_o .

La práctica anterior a Rubin (1976) era el *análisis de casos completos*: descartar toda fila con algún faltante. Su costo se ve con un cálculo: si cada una de p columnas falta de forma independiente con probabilidad q , una fila está completa con probabilidad $(1 - q)^p$, que para $q = 0.05$ y $p = 50$ vale 0.077; se pierde más del 90 % de la muestra. La alternativa, imputar, plantea la pregunta de cuándo lo que se calcula con los valores observados describe correctamente a la población. La respuesta depende de la relación entre R y los datos.

Definición 2.1 (Mecanismos de falta). La falta de X es

- (I) *completamente al azar* (MCAR) si R es independiente de (X, W) ;
- (II) *al azar* (MAR) si R y X son condicionalmente independientes dado W ;
- (III) *no al azar* (MNAR) si no se cumple (ii).

MCAR implica MAR: si R es independiente de (X, W) , entonces $\mathbb{P}(R = 1 \mid X, W) = \mathbb{P}(R = 1)$, que en particular no depende de X una vez fijado W . Ejemplos: una falla aleatoria del sensor es MCAR; que los jóvenes declaren menos su ingreso, con la edad en W , es MAR; que quien gana más declare menos su ingreso es MNAR. La jerarquía importa por lo siguiente.

Proposición 2.2. Supóngase $\mathbb{E}|X| < \infty$ y $\mathbb{P}(R = 0) > 0$.

- (a) Bajo MCAR, $\mathbb{E}[X \mid R = 0] = \mathbb{E}[X]$.
- (b) Bajo MAR, $\mathbb{E}[X \mid W, R = 0] = \mathbb{E}[X \mid W]$ (para los valores de W con $\mathbb{P}(R = 0 \mid W) > 0$).

Demostración. (a) Por la independencia de R y X , $\mathbb{E}[X \mathbf{1}\{R = 0\}] = \mathbb{E}[X] \mathbb{P}(R = 0)$; dividiendo por $\mathbb{P}(R = 0)$ se obtiene $\mathbb{E}[X \mid R = 0] = \mathbb{E}[X]$. (b) Es el mismo argumento condicionado a W : por la independencia condicional, $\mathbb{E}[X \mathbf{1}\{R = 0\} \mid W] = \mathbb{E}[X \mid W] \mathbb{P}(R = 0 \mid W)$, y se divide por $\mathbb{P}(R = 0 \mid W)$. $\square$

La parte (a) dice que bajo MCAR la media de los observados estima sin sesgo la media poblacional; la (b), que bajo MAR la regresión de X sobre W ajustada solo con las filas observadas estima la regresión poblacional, lo que justifica imputar con una predicción a partir de W en lugar de con una constante. Los ejemplos siguientes muestran que las hipótesis no pueden quitarse.

Ejemplo 2.3 (MAR sin MCAR). Sea $W \sim N(0, 1)$, $X = W + \varepsilon$ con $\varepsilon \sim N(0, 1)$ independiente de W , y $R = \mathbf{1}\{W > 0\}$. La falta es MAR (es función de W) pero no MCAR. $\mathbb{E}[X \mid R = 0] = \mathbb{E}[W \mid W \leq 0] = -\varphi(0)/\Phi(0) = -\sqrt{2/\pi} \approx -0.80$, donde φ y Φ son la densidad y la función de distribución normales estándar (se usa $\int_{-\infty}^0 w\varphi(w) dw = -\varphi(0)$, consecuencia de $\varphi'(w) = -w\varphi(w)$). La media de los observados subestima $\mathbb{E}[X] = 0$, pero $\mathbb{E}[X \mid W, R = 0] = W = \mathbb{E}[X \mid W]$, como predice (b).

Ejemplo 2.4 (MNAR). Sea $X \sim N(0, 1)$ observada solo si $X < 1$ (sin W). Entonces $\mathbb{E}[X \mid R = 0] = \mathbb{E}[X \mid X < 1] = -\varphi(1)/\Phi(1) \approx -0.288$ por el mismo cálculo. Los valores observados tienen la distribución de X condicionada a $X < 1$, y sin conocer el mecanismo de falta no hay forma de recuperar $\mathbb{E}[X]$ a partir de ellos: imputar con su media hereda el sesgo.

Definición 2.5 (Imputación simple e indicador). La *imputación por la media* (respectivamente, por la mediana o la moda) reemplaza cada valor faltante de una columna por la media (respectivamente, la mediana o el valor más frecuente) de los valores observados de esa columna en entrenamiento. La *indicadora de falta* es la columna $(r_1, \dots, r_n)$ añadida como característica.

Proposición 2.6 (Efecto de la imputación por la media). Sean $z \in \mathbb{R}^n$ una columna con $n_o \geq 2$ valores observados (índices en O) y $y \in \mathbb{R}^n$ otra columna completa. Sea $\tilde{z}$ la columna imputada con $\bar{z}_o = n_o^{-1} \sum_{k \in O} z_k$. Con varianzas y covarianzas muestrales de divisor $n-1$ para las columnas completas y $n_o - 1$ para las calculadas en O :

- (a) $\bar{\tilde{z}} = \bar{z}_o$;
- (b) $s^2(\tilde{z}) = \frac{n_o-1}{n-1} s_o^2(z)$;
- (c) $s(\tilde{z}, y) = \frac{n_o-1}{n-1} s_o(z, y)$, donde $s_o(z, y)$ es la covarianza de z e y calculada sobre O ;
- (d) $\text{Corr}(\tilde{z}, y) = \sqrt{\frac{n_o-1}{n-1}} \frac{s_o(y)}{s(y)} \text{Corr}_o(z, y)$, donde Corr_o y $s_o(y)$ se calculan sobre O y se supone $s_o(z) > 0$.

Demostración. (a) $\sum_k \tilde{z}_k = \sum_{k \in O} z_k + (n - n_o) \bar{z}_o = n_o \bar{z}_o + (n - n_o) \bar{z}_o$. (b) Por (a), las filas imputadas valen exactamente la media, así que $\sum_k (\tilde{z}_k - \bar{z}_o)^2 = \sum_{k \in O} (z_k - \bar{z}_o)^2 = (n_o - 1) s_o^2(z)$; se divide por $n - 1$. (c) Por lo mismo, $\sum_k (\tilde{z}_k - \bar{z}_o)(y_k - \bar{y}) = \sum_{k \in O} (z_k - \bar{z}_o)(y_k - \bar{y})$. Como $\sum_{k \in O} (z_k - \bar{z}_o) = 0$, reemplazar $\bar{y}$ por la media $\bar{y}_o$ de y sobre O no altera la suma, que vale $(n_o - 1) s_o(z, y)$; se divide por $n - 1$. (d) Se divide (c) por $s(\tilde{z}) s(y)$, con $s(\tilde{z})$ tomada de (b), y se escribe $s_o(z, y) = \text{Corr}_o(z, y) s_o(z) s_o(y)$. $\square$

La imputación por la media conserva la media pero contrae la varianza y la covarianza por el factor $(n_o - 1)/(n - 1)$, sea cual sea el mecanismo. Bajo MCAR, las filas de O son una muestra aleatoria de la población, $s_o(y)$ y $s(y)$ estiman la misma desviación estándar, y (d) muestra que la correlación queda multiplicada por un factor cercano a $\sqrt{n_o/n}$: con 30 % de faltantes, por ≈ 0.84 . En el ejemplo del capítulo, la columna (1, falta, 5) se imputa como (1, 3, 5), con varianza 4 frente a la varianza 8 de los observados: $(2 - 1)/(3 - 1) = 1/2$.

Proposición 2.7 (La indicadora absorbe la media del grupo con faltantes). Sea $\tilde{z}$ la columna imputada con una constante a y considérese la regresión por mínimos cuadrados de y sobre $\mathbf{1}$, $\tilde{z}$ y la indicadora r , con al menos una fila con $r_k = 1$. El valor ajustado en cada fila con $r_k = 1$ es la media de y sobre esas filas, cualquiera que sea el mecanismo de falta.

Demostración. Por el Lema 1.9, el residuo $y - \hat{y}$ es ortogonal a cada columna del diseño, en particular a r : $\sum_k r_k (y_k - \hat{y}_k) = 0$. En las filas con $r_k = 1$ se tiene $\tilde{z}_k = a$, luego $\hat{y}_k = \hat{\beta}_0 + \hat{\beta}_a a + \hat{\gamma}$ es la misma constante para todas ellas; que la suma de sus residuos sea cero obliga a que esa constante sea la media de sus y_k . $\square$

Bajo MNAR, la respuesta media del grupo con faltantes puede diferir de la de cualquier individuo con $z = a$; la indicadora permite al modelo estimarla directamente, al precio de una columna por característica afectada.

Para comparar con la imputación por la mediana conviene el caso de una constante cualquiera.

Proposición 2.8. Con la notación de la Proposición 2.6, sea $\tilde{z}^{(a)}$ la columna imputada con una constante a . Entonces

$$(n - 1) s^2(\tilde{z}^{(a)}) = (n_o - 1) s_o^2(z) + \frac{n_o(n - n_o)}{n} (\bar{z}_o - a)^2.$$

Demostración. La media de $\tilde{z}^{(a)}$ es $\tilde{m} = (n_o \bar{z}_o + (n - n_o)a)/n$, de donde $\bar{z}_o - \tilde{m} = \frac{n - n_o}{n}(\bar{z}_o - a)$ y $a - \tilde{m} = -\frac{n_o}{n}(\bar{z}_o - a)$. Descomponiendo alrededor de $\bar{z}_o$, $\sum_{k \in O} (z_k - \tilde{m})^2 = (n_o - 1)s_o^2 + n_o(\bar{z}_o - \tilde{m})^2$ (el término cruzado se anula porque $\sum_{k \in O} (z_k - \bar{z}_o) = 0$), y las $n - n_o$ filas imputadas aportan $(n - n_o)(a - \tilde{m})^2$. Sumando, $n_o \frac{(n - n_o)^2}{n^2}(\bar{z}_o - a)^2 + (n - n_o) \frac{n_o^2}{n^2}(\bar{z}_o - a)^2 = \frac{n_o(n - n_o)}{n}(\bar{z}_o - a)^2$. $\square$

Con $a = \bar{z}_o$ se recupera la Proposición 2.6(b). Con otra constante, por ejemplo la mediana de una columna asimétrica, aparece un término adicional que no proviene de variación real de los datos: los valores imputados siguen siendo idénticos entre sí y no contienen información sobre y , pero desplazan la media y añaden varianza.

Contraste. La media imputada conserva la media observada (Proposición 2.6(a)) pero, como la media tiene punto de ruptura $1/n$ (Proposición 2.16), un solo atípico entre los observados puede desplazar el valor imputado arbitrariamente; la mediana tiene punto de ruptura cercano a $1/2$. La moda es la única de las tres definida para categóricas. Toda imputación por una constante asigna el mismo valor a filas cuyas respuestas pueden ser distintas, y por eso atenúa la relación con y (Proposición 2.6(d) para la media). Bajo MAR, la imputación condicional a W tiene la justificación de la Proposición 2.2(b); bajo MNAR, ninguna imputación calculada con los observados tiene garantía de insesgadez (Ejemplo 2.4), y la indicadora permite al menos que el modelo estime por separado la respuesta del grupo con faltantes. Algunas implementaciones de *gradient boosting* (XGBoost, LightGBM, `HistGradientBoosting` de scikit-learn) aceptan faltantes: en cada corte eligen con los datos de entrenamiento hacia qué hijo enviarlos, lo que equivale a aprender la indicadora dentro del árbol.

Implementación en AWS. **Data Wrangler**, grupo *Handle missing: Impute* (media o mediana para numéricas, valor más frecuente para categóricas), *Add indicator for missing* (la indicadora de la Proposición 2.7), *Fill missing* (constante dada) y *Drop missing*; es un paso con parámetros ajustados, con *refit*. **Glue DataBrew**: acciones `FILL_WITH_AVERAGE`, `FILL_WITH_MEDIAN`, `FILL_WITH_MOST_FREQUENT`, entre otras. Su documentación indica que el estadístico se calcula sobre todos los valores de la columna del dataset al que se aplica la receta; si la receta se ejecuta por separado sobre prueba, usa la mediana de prueba, lo que viola la separación de la Proposición 1.3. Para respetarla se calcula el valor en entrenamiento y se rellena con ese valor fijo.

```
{"RecipeAction": {"Operation": "FILL_WITH_MEDIAN",
                  "Parameters": {"sourceColumn": "age"}}}
```

SageMaker Processing: `SimpleImputer(strategy=..., add_indicator=True)` de scikit-learn, con `fit` sobre entrenamiento; la imputación condicional de la Proposición 2.2(b) corresponde a `IterativeImputer`. Para series de tiempo, Data Wrangler ofrece además relleno hacia adelante, hacia atrás e interpolación.

2.2. Valores atípicos

Una *regla de detección* asigna a cada columna $z \in \mathbb{R}^n$ el conjunto de índices que declara atípicos. Las reglas del capítulo son de umbral: declaran atípico a z_k cuando una puntuación calculada con z_k y con estadísticos de la columna supera un valor fijo. Se comparan por dos propiedades: si pueden detectar algo con el tamaño de muestra disponible, y si el atípico altera los estadísticos con que se le juzga.

2.2.1. Regla de la puntuación z

Definición 2.9. La puntuación z de z_k es $\zeta_k = (z_k - \bar{z})/s$, con $s > 0$. La regla declara atípico a z_k si $|\zeta_k| > c$, típicamente con $c = 3$.

El valor $c = 3$ se elige porque si $Z \sim N(0, 1)$, $\mathbb{P}(|Z| > 3) \approx 0.0027$. Pero $\bar{z}$ y s se calculan con la propia observación que se juzga, y eso limita la regla.

Proposición 2.10 (Cota de Shiffler). *Para toda columna con $n \geq 2$ y $s > 0$, y todo k , $|\zeta_k| \leq \sqrt{n-1}$ si $s = s_n$, y $|\zeta_k| \leq (n-1)/\sqrt{n}$ si $s = s_{n-1}$.*

Demostración. Sea $d_i = z_i - \bar{z}$. Como $\sum_i d_i = 0$, $d_k = -\sum_{i \neq k} d_i$, y por la desigualdad de Cauchy-Schwarz aplicada a los vectores $(d_i)_{i \neq k}$ y $(1, \dots, 1) \in \mathbb{R}^{n-1}$, $d_k^2 \leq (n-1) \sum_{i \neq k} d_i^2$. Luego $\sum_i d_i^2 \geq d_k^2(1 + \frac{1}{n-1}) = d_k^2 \frac{n}{n-1}$. Si $s = s_n$, $\sum_i d_i^2 = ns^2$ y resulta $\zeta_k^2 \leq n-1$; si $s = s_{n-1}$, $\sum_i d_i^2 = (n-1)s^2$ y resulta $\zeta_k^2 \leq (n-1)^2/n$. $\square$

La igualdad en Cauchy-Schwarz ocurre cuando todos los d_i , $i \neq k$, son iguales, así que la cota se alcanza. Consecuencia: la regla con $c = 3$ no puede declarar atípico a nadie si $\sqrt{n-1} \leq 3$, es decir, si $n \leq 10$ con s_n (el divisor de `scipy.stats.zscore`), ni si $(n-1)/\sqrt{n} \leq 3$, es decir, $n \leq 11$, con s_{n-1} . Esta es la razón aritmética de que el ejemplo del capítulo ($n = 10$) no detecte nada. (La nota del texto base atribuye la cota $(n-1)/\sqrt{n}$ al divisor n ; corresponde al divisor $n-1$. Su conclusión se mantiene.)

El segundo defecto es el *enmascaramiento*: el atípico infla s y reduce su propia puntuación. En el ejemplo, el valor 23 tiene $\zeta = 2.58$ con $\bar{z}$ y s_n calculados con las diez observaciones, pero $\zeta = 5.31$ si se calculan con las otras nueve.

2.2.2. Regla del rango intercuartílico

Definición 2.11 (Vallas de Tukey). Se declara atípico a z_k si $z_k < Q_1 - 1.5 \text{ IQR}$ o $z_k > Q_3 + 1.5 \text{ IQR}$.

Si $Z \sim N(\mu, \sigma^2)$, los cuartiles poblacionales son $\mu \pm 0.6745\sigma$ (porque $\Phi^{-1}(0.75) = 0.6745$), el IQR poblacional es 1.349σ y la valla superior está en $\mu + 2.698\sigma$; la probabilidad de quedar fuera de las vallas es $2(1 - \Phi(2.698)) \approx 0.0070$. La regla equivale entonces, bajo normalidad, a un umbral de 2.7 desviaciones con parámetros calculados de otro modo. La diferencia con la regla z es que esos parámetros no se mueven con el atípico:

Proposición 2.12 (Ausencia de enmascaramiento). *Sea $n \geq 5$ y supóngase que $z_{(n)} > z_{(n-1)}$. Si se reemplaza el máximo por cualquier valor mayor, Q_1 , Q_3 y las vallas no cambian. En particular, al aumentar el máximo acaba siendo declarado atípico.*

Demostración. Para $q = 3/4$, $h = \frac{3}{4}(n-1) + 1$ y $r = \lfloor h \rfloor$. Si $n \geq 5$, $h \leq n - \frac{1}{4}(n-1) \leq n-1$, de modo que $r+1 \leq n$ y, si $r+1 = n$, es porque $h = n-1$ es entero y el peso $h-r$ de $z_{(r+1)}$ es 0. En todos los casos Q_3 depende solo de $z_{(1)}, \dots, z_{(n-1)}$, que no cambian cuando el máximo aumenta (sigue siendo el máximo). Lo mismo vale para Q_1 , con índices menores. Las vallas quedan fijas y el máximo termina por superarlas. $\square$

En el ejemplo del capítulo, $Q_1 = 6.375$ y $Q_3 = 11.75$, así que la valla superior es 19.81 y el valor 23 queda fuera; con 100 o 1000 en su lugar, las vallas son las mismas.

2.2.3. Puntuación z modificada

Definición 2.13 (Iglewicz y Hoaglin, 1993). Con $\text{MAD}(z) = \text{med}(|z_1 - \text{med}(z)|, \dots, |z_n - \text{med}(z)|)$, la puntuación z modificada es $M_k = 0.6745(z_k - \text{med}(z))/\text{MAD}(z)$. Los autores sugieren declarar atípicos los $|M_k| > 3.5$.

El factor 0.6745 hace M_k comparable con ζ_k : si $Z \sim N(\mu, \sigma^2)$, la mediana poblacional de $|Z - \mu|$ es el número a con $\mathbb{P}(|Z - \mu| \leq a) = 1/2$, es decir, $2\Phi(a/\sigma) - 1 = 1/2$, $a = 0.6745\sigma$; así, $\text{MAD}/0.6745$ estima σ . En el ejemplo, $\text{med} = 8.25$, $\text{MAD} = 3$ y $M = 3.32$ para el valor 23: supera el umbral 3 (el que usa DataBrew por defecto) y no el 3.5. Los umbrales son convenciones de los autores y de las herramientas; ninguno es consecuencia de un resultado.

2.2.4. Punto de ruptura

La sensibilidad de un estadístico al atípico se mide con el punto de ruptura (Donoho y Huber, 1983).

Definición 2.14. Sea $T : \mathbb{R}^n \rightarrow \mathbb{R}$ un estadístico y $z \in \mathbb{R}^n$. Su *punto de ruptura* en z es $\varepsilon^*(T, z) = m^*/n$, donde m^* es el menor número de coordenadas de z que, reemplazadas por valores arbitrarios, permiten hacer $|T|$ tan grande como se quiera.

Lema 2.15 (Estadísticos de orden bajo contaminación). Sea $1 \leq r \leq n$ y sea z'_r obtenido de z reemplazando m coordenadas. Si $m \leq \min(r-1, n-r)$, entonces z'_r está entre el mínimo y el máximo de las coordenadas de z no reemplazadas. Si $m \geq n-r+1$, z'_r puede hacerse arbitrariamente grande; si $m \geq r$, arbitrariamente negativo.

Demostración. Sean a y b el mínimo y el máximo de las $n-m$ coordenadas no reemplazadas. Hay al menos $n-m \geq r$ coordenadas de z' menores o iguales que b , de modo que el r -ésimo menor valor es $\leq b$. Hay a lo sumo $m \leq r-1$ coordenadas menores que a , de modo que el r -ésimo menor valor es $\geq a$. Recíprocamente, si $m \geq n-r+1$ y los valores reemplazados se fijan en $M > b$, hay a lo sumo $n-m \leq r-1$ coordenadas menores que M , luego $z'_r = M$; el caso $m \geq r$ es simétrico. $\square$

Proposición 2.16. Para todo z : la media, s_n , s_{n-1} , el máximo, el mínimo y $\max_k |z_k|$ tienen punto de ruptura $1/n$; la mediana tiene punto de ruptura $\lceil n/2 \rceil/n$, que tiende a $1/2$; Q_1 , Q_3 y el IQR tienen punto de ruptura que tiende a $1/4$ cuando $n \rightarrow \infty$.

Demostración. Reemplácese z_1 por M . La nueva media es $\bar{z}' = (M + \sum_{k \geq 2} z_k)/n \rightarrow \infty$ cuando $M \rightarrow \infty$. Como la suma de cuadrados contiene el sumando $(M - \bar{z}')^2$ y $M - \bar{z}' = \frac{n-1}{n}M - \frac{1}{n} \sum_{k \geq 2} z_k \rightarrow \infty$, se tiene $s_n \geq |M - \bar{z}'|/\sqrt{n} \rightarrow \infty$, y lo mismo para $s_{n-1} \geq s_n$. El máximo y $\max_k |z_k|$ son $\geq M$; con $-M$ en lugar de M , el mínimo es $\leq -M$. Así, $m^* = 1$.

Por la definición del cuantil, $Q(q)$ es una combinación convexa de $z_{(r)}$ y $z_{(r+1)}$ con $r = \lfloor (n-1)q + 1 \rfloor$, y está acotado si ambos lo están. Para la mediana con n impar, $Q(1/2) = z_{((n+1)/2)}$ y el Lema 2.15 con $r = (n+1)/2$ da $m^* = n - r + 1 = (n+1)/2 = \lceil n/2 \rceil$. Con n par, $Q(1/2) = \frac{1}{2}(z_{(n/2)} + z_{(n/2+1)})$; el lema da acotamiento para $m \leq n/2 - 1$ y explosión de $z_{(n/2+1)}$ con $m = n/2$, así que $m^* = n/2$. Para Q_3 , $r = \lfloor \frac{3}{4}(n-1) + 1 \rfloor$ y la explosión requiere $m \geq n-r$ o $n-r+1$ según el peso de $z_{(r+1)}$; en ambos casos $m^*/n \rightarrow 1/4$. Para Q_1 , con $r = \lfloor \frac{1}{4}(n-1) + 1 \rfloor$, la explosión hacia $-\infty$ requiere $m \geq r$, y $r/n \rightarrow 1/4$. Si ambos cuartiles quedan acotados, su diferencia también; y como $\text{IQR} = Q_3 - Q_1 \geq 0$, el IQR crece sin límite cuando $Q_3 \rightarrow \infty$ con Q_1 acotado o $Q_1 \rightarrow -\infty$ con Q_3 acotado. Su punto de ruptura es entonces el menor de los de los cuartiles, que tiende a $1/4$. $\square$

Tratamientos y contraste. Ante una observación declarada atípica caben: eliminar la fila (correcto si el valor es un error de registro, porque entonces no describe la población); *recortar* (*winsorizar*), reemplazando z_k por $\min(\max(z_k, l), u)$ con l, u las vallas; imputar como si faltara; o transformar toda la columna (§4.1). La comparación entre reglas se sigue de los resultados anteriores: la regla z requiere $n > c^2 + 1$ para poder detectar algo (Proposición 2.10) y sufre enmascaramiento porque sus parámetros tienen punto de ruptura $1/n$; las vallas de Tukey y la z modificada usan estadísticos con punto de ruptura $1/4$ y $1/2$ y no sufren enmascaramiento por un solo valor (Proposición 2.12). Ninguna regla aplicada columna por columna detecta atípicos conjuntos: en los puntos $(-2, -2)$, $(-1, -1)$, $(0, 0)$, $(1, 1)$, $(2, 2)$, $(2, -2)$, el último rompe la relación $x_2 = x_1$, pero su valor 2 en la primera columna también lo tiene $(2, 2)$ y su valor -2 en la segunda también lo tiene $(-2, -2)$; una regla de umbral que marcara esos valores marcaría también esas filas normales.

Implementación en AWS. **Glue DataBrew:** FLAG_OUTLIERS, REMOVE_OUTLIERS, REPLACE_OUTLIERS, RESCALE_OUTLIERS_WITH_Z_SCORE y RESCALE_OUTLIERS_WITH_SKEW, con

outlierStrategy igual a Z_SCORE ($|\zeta_k|$), MODIFIED_Z_SCORE ($|M_k|$) o IQR (vallas con el múltiplo dado), y threshold con valor por defecto 3:

```
{"Action": {"Operation": "FLAG_OUTLIERS",
  "Parameters": {"sourceColumn": "energia", "targetColumn": "energia_atipico",
    "outlierStrategy": "IQR", "threshold": "1.5",
    "trueString": "Yes", "falseString": "No"}}}
```

Data Wrangler, grupo *Handle outliers: Standard deviation numeric outliers* (regla z), *Robust standard deviation numeric outliers* (centro y escala calculados entre dos cuantiles), *Quantile numeric outliers* y *Min-max numeric outliers* (umbrales dados), cada uno con corrección *Clip* (recortar), *Remove* o *Invalidate* (convertir en faltante). Para series de tiempo, el análisis *Anomaly detection* descompone la serie en tendencia, estacionalidad y residuo y marca los residuos que superan un múltiplo dado de su desviación estándar.

3. Características numéricas: escalamiento lineal

3.1. Definición y casos

Definición 3.1 (Escalamiento lineal). Sea $z \in \mathbb{R}^n$ la columna de entrenamiento de una característica numérica. Un *escalamiento lineal* es la transformación ajustada $T_{a,b}(u) = (u - a)/b$, $u \in \mathbb{R}$, con parámetros $a \in \mathbb{R}$ (centro) y $b > 0$ (escala) calculados con z . Los casos del capítulo son:

Nombre	a	b
Estandarización (puntuación z)	$\bar{z}$	$s_n(z)$
Normalización MinMax	$z_{(1)}$	$z_{(n)} - z_{(1)}$
Escalado robusto	$\text{med}(z)$	$\text{IQR}(z)$
Escalado MaxAbs	0	$\max_k z_k $
Normalización por la media	$\bar{z}$	$z_{(n)} - z_{(1)}$

La condición $b > 0$ tiene dos funciones. Excluye $b = 0$, que haría T indefinida; y, frente a $b < 0$, garantiza que T sea creciente y conserve el orden de los valores. No todos los casos la cumplen en las mismas columnas: $s_n(z) > 0$ y $z_{(n)} - z_{(1)} > 0$ si y solo si z no es constante, y $\max |z_k| > 0$ si y solo si z no es nula; en cambio, el IQR puede anularse en columnas no constantes.

Contraejemplo 3.2. Sea $z = (0, 0, 0, 0, 0, 0, 0, 5)$, $n = 8$. Para $q = 1/4$, $h = 2.75$ y $Q_1 = z_{(2)} + 0.75(z_{(3)} - z_{(2)}) = 0$; para $q = 3/4$, $h = 6.25$ y $Q_3 = z_{(6)} + 0.25(z_{(7)} - z_{(6)}) = 0$. Luego $\text{IQR} = 0$ aunque la columna no es constante. En general, por la definición del cuantil, $\text{IQR} = 0$ si y solo si todos los estadísticos de orden que intervienen en Q_1 y Q_3 (y por tanto los intermedios) son iguales: un mismo valor ocupa las posiciones centrales de la muestra ordenada, como en conteos con muchos ceros. scikit-learn reemplaza entonces la escala por 1.

Sobre la muestra de entrenamiento, MinMax toma valores en $[0, 1]$ y MaxAbs en $[-1, 1]$, porque $z_{(1)} \leq z_k \leq z_{(n)}$ y $|z_k| \leq \max_i |z_i|$. Aplicada a un valor nuevo u fuera de $[z_{(1)}, z_{(n)}]$, MinMax da un resultado fuera de $[0, 1]$: los parámetros son los de entrenamiento (Proposición 1.3). La normalización por la media da media 0 y valores en $[-1, 1]$ (porque $|z_k - \bar{z}| \leq z_{(n)} - z_{(1)}$), pero su desviación estándar es $s_n/(z_{(n)} - z_{(1)})$, que no es 1 en general.

3.2. Qué cambia y qué no cambia

Para $z \in \mathbb{R}^n$ sean los momentos centrales $m_r(z) = n^{-1} \sum_k (z_k - \bar{z})^r$, el *coeficiente de asimetría* $\gamma_1(z) = m_3/m_2^{3/2}$ y la *curtosis* $\gamma_2(z) = m_4/m_2^2$ (definidos si $m_2 > 0$).

Proposición 3.3 (Invariancia de la forma). Si $b > 0$, $\gamma_1(T_{a,b}(z)) = \gamma_1(z)$ y $\gamma_2(T_{a,b}(z)) = \gamma_2(z)$, donde $T_{a,b}(z)$ se aplica coordenada a coordenada. Además, la correlación de Pearson entre $T_{a,b}(z)$ y cualquier otra columna w coincide con la de z y w .

Demostración. La media de $T_{a,b}(z)$ es $(\bar{z} - a)/b$, así que sus desviaciones respecto de la media son $(z_k - \bar{z})/b$ y $m_r(T_{a,b}(z)) = b^{-r}m_r(z)$. Entonces $\gamma_1(T_{a,b}(z)) = b^{-3}m_3/(b^{-2}m_2)^{3/2} = m_3/m_2^{3/2}$, donde se usó $(b^{-2})^{3/2} = b^{-3}$, válido porque $b > 0$; para γ_2 , $b^{-4}m_4/(b^{-2}m_2)^2 = m_4/m_2^2$. La covarianza de $T_{a,b}(z)$ con w es la de z con w multiplicada por $1/b$, y la desviación estándar de $T_{a,b}(z)$ es la de z multiplicada por $1/b$; el factor se cancela en el cociente. $\square$

Si $b < 0$ el mismo cálculo da $\gamma_1(T_{a,b}(z)) = -\gamma_1(z)$: la hipótesis $b > 0$ es necesaria para la invariancia de la asimetría. La proposición es el fundamento de la afirmación del capítulo de que ningún escalado lineal corrige la asimetría.

Proposición 3.4 (Compresión por un valor extremo en MinMax). Fíjense $z_1, \dots, z_{n-1}$ y sea $z_n = M$. Cuando $M \rightarrow \infty$, la normalización MinMax de cada z_k , $k < n$, tiende a 0.

Demostración. Si $M > \max_{k < n} z_k$, entonces $z_{(n)} = M$ y $z_{(1)} = \min_{k < n} z_k$ no depende de M ; el valor normalizado $(z_k - z_{(1)})/(M - z_{(1)})$ tiene numerador fijo y denominador que tiende a infinito. $\square$

En la columna de energía del capítulo, sin el valor 23 los otros nueve ocupan $[0, 1]$ tras MinMax; con él, ocupan $[0, 0.426]$. El fenómeno se explica con la Proposición 2.16: los parámetros de MinMax, MaxAbs y la estandarización tienen punto de ruptura $1/n$, los del escalado robusto $\approx 1/4$. El escalado robusto no elimina el valor extremo: por ser creciente, lo conserva como el mayor (en el ejemplo vale 2.74 y el siguiente 0.79); lo que protege es la escala a la que quedan los demás.

Definición 3.5. Una matriz $A \in \mathbb{R}^{n \times p}$ es *dispersa* cuando la mayoría de sus entradas son nulas; un *formato disperso* almacena solo las entradas no nulas con sus posiciones, con memoria proporcional a su número $\text{nnz}(A)$ en lugar de a np .

Un ejemplo es la matriz documento-término de §10.4, donde la entrada (d, t) cuenta las apariciones del término t en el documento d : cada documento usa una fracción ínfima del vocabulario.

Proposición 3.6 (Conservación de la dispersión). $T_{a,b}(0) = 0$ si y solo si $a = 0$. Si $a \neq 0$ y la columna tiene n_0 ceros, la columna transformada tiene n_0 entradas iguales a $-a/b \neq 0$. En la Definición 3.1, solo MaxAbs tiene $a = 0$ para toda columna.

Demostración. $T_{a,b}(0) = -a/b$, nulo si y solo si $a = 0$ porque $b > 0$. En los demás casos de la tabla, a es la media, el mínimo o la mediana, que en general son no nulos (por ejemplo, la media de una columna no negativa y no nula es positiva). $\square$

Con 10^6 documentos, 10^5 términos y 100 términos distintos por documento, la matriz tiene 10^8 entradas no nulas; centrarla produce 10^{11} . Dividir por s_n sin centrar ($a = 0$, `StandardScaler(with_mean=False)`) también conserva la dispersión.

3.3. Escala y descenso de gradiente

El capítulo afirma que escalar acelera la convergencia del descenso de gradiente. La afirmación se precisa para mínimos cuadrados.

Proposición 3.7. Sea $f(\beta) = \frac{1}{2n}\|y - X\beta\|^2$, $X \in \mathbb{R}^{n \times p}$, y supóngase que $H = X^\top X/n$ es definida positiva, con autovalores $\lambda_{\max} \geq \dots \geq \lambda_{\min} > 0$; sea $\kappa = \lambda_{\max}/\lambda_{\min}$ (número de condición). Entonces f tiene un único minimizador $\hat{\beta}$, y el descenso de gradiente $\beta_{t+1} = \beta_t - \eta \nabla f(\beta_t)$ con paso $\eta = 1/\lambda_{\max}$ cumple, para todo β_0 , $\|\beta_t - \hat{\beta}\| \leq (1 - 1/\kappa)^t \|\beta_0 - \hat{\beta}\|$.

Demostración. $\nabla f(\beta) = H\beta - X^\top y/n$; como H es invertible, ∇f se anula solo en $\hat{\beta} = H^{-1}X^\top y/n$, y como la hessiana H es definida positiva, f es estrictamente convexa y $\hat{\beta}$ es su único minimizador. Usando $X^\top y/n = H\hat{\beta}$, $\nabla f(\beta) = H(\beta - \hat{\beta})$, y $\beta_{t+1} - \hat{\beta} = (I - \eta H)(\beta_t - \hat{\beta})$. La matriz $I - H/\lambda_{\max}$ es simétrica con autovalores $1 - \lambda_i/\lambda_{\max} \in [0, 1 - 1/\kappa]$, de modo que su norma espectral es $1 - 1/\kappa$ y $\|\beta_{t+1} - \hat{\beta}\| \leq (1 - 1/\kappa)\|\beta_t - \hat{\beta}\|$; se itera. $\square$

Supóngase dos características con media cero y covarianza muestral nula, de varianzas s_1^2 y s_2^2 (y la respuesta centrada, de modo que no hace falta intercepto). Entonces $H = \text{diag}(s_1^2, s_2^2)$ y $\kappa = s_1^2/s_2^2$ si $s_1 \geq s_2$. Si ambas características son longitudes con la misma dispersión, pero una se registra en metros ($s_2 = 1$) y la otra en centímetros ($s_1 = 100$), $\kappa = 10^4$ y la cota garantiza una reducción del error de solo $1 - 10^{-4}$ por iteración. Tras estandarizar, H es la matriz de correlaciones muestrales; con dos características de correlación ρ , sus autovalores son $1 \pm \rho$ (los de $\begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$), con autovectores $(1, \pm 1)$ y $\kappa = (1 + |\rho|)/(1 - |\rho|)$, que depende de la redundancia entre las características y no de sus unidades.

Contraste. Ningún escalamiento lineal cambia la forma de la distribución (Proposición 3.3); la elección depende de la propiedad requerida: rango acotado conocido en entrenamiento (Min-Max), varianzas iguales para el condicionamiento y para que una penalización trate igual a todos los coeficientes (estandarización, Proposición 3.7 y Contraejemplo 1.11), parámetros poco afectados por valores extremos (robusto, con la salvedad de columnas con IQR = 0), conservación de la dispersión (MaxAbs, Proposición 3.6). Para árboles y regresión lineal sin penalizar el escalamiento no cambia las predicciones (Proposiciones 1.7 y 1.10).

Implementación en AWS. Data Wrangler, *Process numeric* $\rightarrow$ *Scale values*, con *Standard scaler*, *Robust scaler*, *Min max scaler* (rango configurable) y *Max absolute scaler*; opera sobre una columna por paso. **Glue DataBrew**, acción SCALE (en el JSON la operación se llama NORMALIZATION) con *strategy* igual a MIN_MAX, SCALE_BETWEEN (a un rango dado), MEAN_NORMALIZATION o Z_SCORE:

```
{"Action": {"Operation": "NORMALIZATION",
  "Parameters": {"sourceColumn": "all_votes", "strategy": "MIN_MAX",
    "targetColumn": "all_votes_normalized"}}
```

La documentación de DataBrew describe MEAN_NORMALIZATION como «media 0 y desviación 1 en $[-1, 1]$ »; con la fórmula de la Definición 3.1 la media es 0 y los valores están en $[-1, 1]$, pero la desviación estándar no es 1 en general. Como toda acción de DataBrew, calcula sus estadísticos sobre el dataset al que se aplica la receta. **SageMaker Processing: StandardScaler, RobustScaler, MinMaxScaler y MaxAbsScaler** de scikit-learn, con *fit* sobre entrenamiento.

4. Características numéricas: transformaciones no lineales

Las transformaciones de esta sección son funciones estrictamente crecientes aplicadas a toda la columna. Por la Proposición 1.7 no alteran los árboles; su efecto recae sobre modelos lineales, métodos de distancia y redes neuronales. Se estudian tres propósitos distintos: linealizar relaciones, estabilizar la varianza y reducir la asimetría.

4.1. Transformación logarítmica

Linealización. Si $y = ae^{bx}\varepsilon$ con $a > 0$ y un error multiplicativo $\varepsilon > 0$ con $\mathbb{E}[\log \varepsilon] = 0$, entonces $\log y = \log a + bx + \log \varepsilon$: un modelo lineal para $\log y$ con error aditivo de media cero. Si $y = ax^b\varepsilon$ con $x > 0$, la relación es lineal entre $\log y$ y $\log x$. En el ejemplo del capítulo, $y = 10^{x-3}$ da $\log_{10} y = x - 3$.

Estabilización de la varianza. Muchas medidas positivas (ingresos, tiempos, concentraciones) tienen desviación estándar aproximadamente proporcional a su media. El modelo exacto de esa situación es una familia de escala.

Definición 4.1. Sea $G > 0$ una variable aleatoria con distribución fija. La familia de escala generada por G es la familia de distribuciones de $X = \theta G$, $\theta > 0$.

Son familias de escala la exponencial, la gamma con parámetro de forma fijo y la lognormal con σ fijo (si $\log G \sim N(0, \sigma^2)$, entonces $\log(\theta G) \sim N(\log \theta, \sigma^2)$).

Proposición 4.2. Si $X = \theta G$ con $\mathbb{E}[G^2] < \infty$ y $\mathbb{E}[(\log G)^2] < \infty$, entonces $\mathbb{E}[X] = \theta \mathbb{E}[G]$ y $\sqrt{\text{Var } X} = \theta \sqrt{\text{Var } G}$, de modo que el cociente desviación/media no depende de θ ; y $\text{Var}(\log X) = \text{Var}(\log G)$ no depende de θ .

Demostración. Las dos primeras igualdades son la linealidad de la esperanza y $\text{Var}(\theta G) = \theta^2 \text{Var } G$. Para la tercera, $\log X = \log \theta + \log G$, y sumar la constante $\log \theta$ no cambia la varianza. $\square$

Es decir: si varios grupos difieren solo en la escala, en la escala original sus varianzas son proporcionales a sus medias al cuadrado, y tras el logaritmo son iguales. Así, con G gamma de forma 3, $\text{Var}(\log X) = \psi'(3) \approx 0.395$ para cualquier θ (ψ' es la función trigamma).

Asimetría. Si X es lognormal, $\log X$ es exactamente normal, simétrica. Que el logaritmo reduzca la asimetría no es, sin embargo, una propiedad general:

Contraejemplo 4.3. Si U es uniforme en $(0, 1)$ (asimetría 0), entonces $-\log U$ es exponencial de media 1, porque $\mathbb{P}(-\log U > t) = \mathbb{P}(U < e^{-t}) = e^{-t}$ para $t \geq 0$. La exponencial de media 1 tiene varianza 1 y tercer momento central 2, luego asimetría 2; por tanto $\log U$ tiene asimetría -2 . El logaritmo convirtió una variable simétrica en una muy asimétrica hacia la izquierda.

El logaritmo comprime más los valores grandes que los pequeños (su derivada $1/x$ decrece), así que acorta colas derechas largas; aplicado a una variable sin esa cola, estira la izquierda. Su uso se justifica en variables con asimetría positiva o de la forma de la Proposición 4.2, y conviene comprobar la asimetría resultante.

Proposición 4.4 (Sesgo de retransformación). Si $\log Y \sim N(\mu, \sigma^2)$ con $\sigma > 0$, entonces $\mathbb{E}[Y] = e^{\mu + \sigma^2/2} > e^\mu$, y e^μ es la mediana de Y .

Demostración. $\mathbb{E}[Y] = \mathbb{E}[e^W]$ con $W \sim N(\mu, \sigma^2)$ es la función generadora de momentos de W , $M_W(t) = e^{\mu t + \sigma^2 t^2/2}$, en $t = 1$. Como $\exp$ es creciente, $\mathbb{P}(Y \leq e^\mu) = \mathbb{P}(W \leq \mu) = 1/2$. $\square$

Si un modelo se ajusta para $\log y$ y se reporta $\exp$ de su predicción, se estima la mediana de Y , no su media. Bajo el modelo lognormal, la media se recupera multiplicando por $e^{\hat{\sigma}^2/2}$, con $\hat{\sigma}^2$ la varianza residual estimada.

Ceros. El logaritmo exige valores positivos. El desplazamiento $\log(1+x)$ (**log1p**) admite ceros, pero su efecto depende de la unidad de medida de x : para $u \geq 0$, $0 \leq u - \log(1+u) \leq u^2/2$ (la función $h(u) = u - \log(1+u)$ cumple $h(0) = 0$ y $0 \leq h'(u) = u/(1+u) \leq u$). Si x se mide en una unidad c veces mayor, $\log(1+x/c)$ difiere de la transformación lineal x/c en a lo sumo $(x/c)^2/2$, que es despreciable cuando x/c es pequeño: con la unidad «grande», **log1p** es casi lineal y no reduce la asimetría. La constante de desplazamiento debe elegirse en relación con la escala de los datos.

4.2. Raíces cuadrada y cúbica

Para conteos, el modelo de referencia es Poisson, cuya varianza es igual a su media; la raíz cuadrada estabiliza esa varianza. El resultado se obtiene con el método delta.

Proposición 4.5 (Método delta). Sean T_n variables aleatorias con $\sqrt{n}(T_n - \mu) \xrightarrow{d} N(0, \sigma^2)$ y g derivable en μ . Entonces $\sqrt{n}(g(T_n) - g(\mu)) \xrightarrow{d} N(0, g'(\mu)^2 \sigma^2)$.

Demostración. Sea $\rho(t) = \frac{g(t) - g(\mu)}{t - \mu} - g'(\mu)$ para $t \neq \mu$ y $\rho(\mu) = 0$; por la definición de derivada, ρ es continua en μ . Se tiene $\sqrt{n}(g(T_n) - g(\mu)) = (g'(\mu) + \rho(T_n))\sqrt{n}(T_n - \mu)$. Como $\sqrt{n}(T_n - \mu)$ converge en distribución, está acotada en probabilidad, y $T_n - \mu = n^{-1/2}\sqrt{n}(T_n - \mu) \rightarrow 0$ en probabilidad; por la continuidad de ρ en μ , $\rho(T_n) \rightarrow 0$ en probabilidad. El teorema de Slutsky da que el producto converge en distribución a $g'(\mu)$ por una $N(0, \sigma^2)$. $\square$

Corolario 4.6. Si $X_\mu \sim \text{Poisson}(\mu)$ con $\mu \in \mathbb{N}$, entonces $\sqrt{X_\mu} - \sqrt{\mu} \xrightarrow{d} N(0, 1/4)$ cuando $\mu \rightarrow \infty$.

Demostración. La suma de variables de Poisson independientes es de Poisson con la suma de los parámetros, así que X_μ tiene la distribución de $\sum_{i=1}^\mu V_i$ con V_i Poisson(1) independientes, de media y varianza 1. Por el teorema central del límite, $\sqrt{\mu}(X_\mu/\mu - 1) \xrightarrow{d} N(0, 1)$. La Proposición 4.5 con $n = \mu$, $T_n = X_\mu/\mu$ y $g = \sqrt{\cdot}$ (con $g'(1) = 1/2$) da $\sqrt{\mu}(\sqrt{X_\mu/\mu} - 1) \xrightarrow{d} N(0, 1/4)$, y $\sqrt{\mu}\sqrt{X_\mu/\mu} = \sqrt{X_\mu}$. $\square$

La varianza de X_μ es μ ; la de $\sqrt{X_\mu}$ se aproxima a $1/4$ sin importar μ (por simulación, 0.252 con $\mu = 50$ y 0.286 con $\mu = 5$; Anscombe mostró que $\sqrt{x + 3/8}$ mejora la aproximación con μ pequeño). La raíz cúbica $x^{1/3}$ está definida en todo $\mathbb{R}$, es impar y estrictamente creciente, así que se aplica a datos con signo sin desplazamientos.

La intensidad con que estas funciones comprimen los valores grandes se ordena por su crecimiento: para $0 < \lambda_1 < \lambda_2$, $x^{\lambda_1}/x^{\lambda_2} = x^{\lambda_1 - \lambda_2} \rightarrow 0$, y para todo $\lambda > 0$, $\log x/x^\lambda \rightarrow 0$ cuando $x \rightarrow \infty$ (por la regla de L'Hôpital, el cociente de derivadas es $1/(\lambda x^\lambda)$). Así, para valores grandes, $\log$ crece más despacio que $\sqrt[3]{\cdot}$, y esta más despacio que $\sqrt{\cdot}$: es el orden de intensidad que da el capítulo («escalera de potencias» de Tukey).

4.3. Transformación de Box–Cox

Las transformaciones anteriores se eligen de antemano. Box y Cox (1964) plantearon elegir la potencia con los datos, dentro de una familia que contiene a $\log$, $\sqrt{\cdot}$ y $1/x$ salvo constantes afines.

Definición 4.7 (Box–Cox). Para $x > 0$ y $\lambda \in \mathbb{R}$, $x^{(\lambda)} = (x^\lambda - 1)/\lambda$ si $\lambda \neq 0$ y $x^{(0)} = \log x$.

Proposición 4.8. Para cada $x > 0$, $\lambda \mapsto x^{(\lambda)}$ es continua en $\lambda = 0$. Para cada λ , $x \mapsto x^{(\lambda)}$ es estrictamente creciente en $(0, \infty)$.

Demostración. $(x^\lambda - 1)/\lambda = (e^{\lambda \log x} - e^0)/\lambda$ es el cociente incremental en 0 de la función $\lambda \mapsto e^{\lambda \log x}$, cuya derivada en 0 es $\log x$; luego el límite cuando $\lambda \rightarrow 0$ es $\log x = x^{(0)}$. La derivada respecto de x es $x^{\lambda-1} > 0$ (para $\lambda = 0$, $1/x$). $\square$

Ambas propiedades justifican la forma de la definición: sin restar 1 y dividir por λ , x^λ sería decreciente para $\lambda < 0$ (invertiría el orden) y constante para $\lambda = 0$, y la familia no variaría con continuidad. La restricción $x > 0$ es necesaria: x^λ no es real para $x < 0$ y λ no entero, y $\log 0$ no existe. Los valores $\lambda = 1, 1/2, 0$ y -1 corresponden, salvo transformaciones afines, a la identidad, la raíz cuadrada, el logaritmo y el recíproco.

Proposición 4.9 (Estimación de λ). *Supóngase como modelo de trabajo que existen λ, μ y $\sigma^2 > 0$ tales que $x_1^{(\lambda)}, \dots, x_n^{(\lambda)}$ son i.i.d. $N(\mu, \sigma^2)$. La log-verosimilitud de λ , maximizada en (μ, σ^2) para cada λ (log-verosimilitud perfilada), es, salvo una constante,*

$$\ell_p(\lambda) = -\frac{n}{2} \log \hat{\sigma}^2(\lambda) + (\lambda - 1) \sum_{k=1}^n \log x_k,$$

donde $\hat{\sigma}^2(\lambda)$ es la varianza, con divisor n , de los $x_k^{(\lambda)}$. Si todos los x_k se multiplican por una constante $c > 0$, ℓ_p cambia en la constante $-n \log c$ y su maximizador $\hat{\lambda}$ no cambia.

Demostración. Por la Proposición 4.8, $u \mapsto u^{(\lambda)}$ es diferenciable y estrictamente creciente, con derivada $u^{\lambda-1}$; por la fórmula de cambio de variable, la densidad de cada x_k es la normal evaluada en $x_k^{(\lambda)}$ multiplicada por $x_k^{\lambda-1}$. La log-verosimilitud es

$$-\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_k (x_k^{(\lambda)} - \mu)^2 + (\lambda - 1) \sum_k \log x_k.$$

Para λ fijo, es la log-verosimilitud normal de los $x_k^{(\lambda)}$ más un término que no depende de (μ, σ^2) , así que se maximiza en la media y la varianza muestrales (divisor n) de los $x_k^{(\lambda)}$; con esos valores, el término cuadrático vale $-n/2$, y queda $\ell_p(\lambda)$ más constantes.

Para la invariancia: si $\lambda \neq 0$, $(cx)^{(\lambda)} = c^\lambda x^{(\lambda)} + (c^\lambda - 1)/\lambda$, y si $\lambda = 0$, $\log(cx) = \log x + \log c$; en ambos casos una transformación afín de $x^{(\lambda)}$ con pendiente c^λ , luego $\hat{\sigma}_c^2(\lambda) = c^{2\lambda} \hat{\sigma}^2(\lambda)$. Además $\sum_k \log(cx_k) = n \log c + \sum_k \log x_k$. Sustituyendo, $\ell_{p,c}(\lambda) = \ell_p(\lambda) - n\lambda \log c + (\lambda - 1)n \log c = \ell_p(\lambda) - n \log c$. $\square$

Dos observaciones delimitan el resultado. Primero, el modelo de trabajo no puede cumplirse exactamente si $\lambda \neq 0$: para $\lambda > 0$, $x^{(\lambda)} > -1/\lambda$, y una normal pone probabilidad positiva debajo de cualquier número; Box y Cox lo presentan como una aproximación, y $\hat{\lambda}$ se interpreta como la potencia que más acerca los datos a la normalidad dentro de la familia. Segundo, el término $(\lambda - 1) \sum \log x_k$, que proviene del cambio de variable, es indispensable: sin él se compararían varianzas medidas en escalas distintas. Por ejemplo, si todos los $x_k > 1$ y $\lambda \rightarrow -\infty$, $x_k^\lambda \rightarrow 0$ y $x_k^{(\lambda)} = -1/\lambda + x_k^\lambda/\lambda$, cuya varianza $\text{Var}(x^\lambda)/\lambda^2$ tiende a 0: minimizar $\hat{\sigma}^2(\lambda)$ sin el término elegiría siempre un λ extremo. Por último, el estimador es invariante a la escala pero no a la traslación: $\hat{\lambda}$ calculado para $x + c$ difiere en general, así que desplazar datos para volverlos positivos cambia el resultado.

Contraejemplo 4.10. Ninguna transformación de la familia (ni ninguna función continua y estrictamente creciente) vuelve normal a una variable cuyo soporte tenga un hueco. Sea X con densidad positiva solo en $[1, 2] \cup [10, 11]$ y g continua y estrictamente creciente. Entonces $g(X) \in [g(1), g(2)] \cup [g(10), g(11)]$ y $\mathbb{P}(g(2) < g(X) < g(10)) = \mathbb{P}(2 < X < 10) = 0$, mientras que una normal asigna probabilidad positiva al intervalo no vacío $(g(2), g(10))$.

Si la función de distribución F de X es continua, la transformación cuantil $\Phi^{-1} \circ F$ sí produce una $N(0, 1)$ exacta: para $u \in (0, 1)$ sea x_u el mayor x con $F(x) = u$ (existe por la continuidad de F y porque $F(x) \rightarrow 1$); como F es no decreciente, $F(X) \leq u$ si y solo si $X \leq x_u$, luego $\mathbb{P}(F(X) \leq u) = F(x_u) = u$, es decir, $F(X)$ es uniforme en $(0, 1)$, y $\mathbb{P}(\Phi^{-1}(F(X)) \leq t) = \mathbb{P}(F(X) \leq \Phi(t)) = \Phi(t)$. El precio es estimar F completa (en la práctica, la función de distribución empírica de entrenamiento), frente a un solo parámetro λ .

4.4. Transformación de Yeo–Johnson

Yeo y Johnson (2000) extendieron la idea a datos con ceros y negativos, que Box–Cox no admite.

Definición 4.11 (Yeo–Johnson). Para $x \in \mathbb{R}$ y $\lambda \in \mathbb{R}$,

$$\psi(\lambda, x) = \begin{cases} ((x+1)^\lambda - 1)/\lambda, & x \geq 0, \lambda \neq 0, \\ \log(x+1), & x \geq 0, \lambda = 0, \\ -((1-x)^{2-\lambda} - 1)/(2-\lambda), & x < 0, \lambda \neq 2, \\ -\log(1-x), & x < 0, \lambda = 2. \end{cases}$$

Proposición 4.12. Para cada λ , $x \mapsto \psi(\lambda, x)$ es derivable en $\mathbb{R}$ con derivada positiva: $(x+1)^{\lambda-1}$ si $x \geq 0$ y $(1-x)^{1-\lambda}$ si $x < 0$. Además $\psi(\lambda, 0) = 0$, para $x \geq 0$ coincide con la Box–Cox de $x+1$, y $\psi(1, x) = x$.

Demostración. Las derivadas de cada rama se calculan directamente (también en los casos logarítmicos: $1/(x+1)$ y $1/(1-x)$, que coinciden con las fórmulas en $\lambda = 0$ y $\lambda = 2$); son positivas porque $x+1 > 0$ para $x \geq 0$ y $1-x > 0$ para $x < 0$. En $x = 0$ ambas ramas valen 0 y sus derivadas laterales valen 1, luego ψ es derivable en 0. La coincidencia con Box–Cox es la definición de la rama $x \geq 0$. Con $\lambda = 1$: $(x+1) - 1 = x$ si $x \geq 0$ y $-((1-x) - 1) = x$ si $x < 0$. $\square$

La estimación de λ sigue la Proposición 4.9 con el término de cambio de variable $(\lambda - 1) \sum_k \text{sgn}(x_k) \log(|x_k| + 1)$, que resulta de sumar los logaritmos de las derivadas anteriores. En el ejemplo del capítulo (datos simétricos), scikit-learn estima $\hat{\lambda} \approx 0.92$ y 1.02 , cercanos al valor 1 de la identidad.

Contraste.

	Dominio	Parámetro	Situación que la justifica
$\log x$	$x > 0$	ninguno	familia de escala (Prop. 4.2); relación multiplicativa
$\log(1 + x/c)$	$x > -c$	c , elegido según la unidad	ídem con ceros
$\sqrt{x}$	$x \geq 0$	ninguno	conteos de Poisson (Cor. 4.6)
$\sqrt[3]{x}$	$\mathbb{R}$	ninguno	asimetría con valores de ambos signos
Box–Cox	$x > 0$	λ por máxima verosimilitud	asimetría de grado desconocido
Yeo–Johnson	$\mathbb{R}$	λ por máxima verosimilitud	ídem, con ceros y negativos
$\Phi^{-1} \circ \hat{F}$	$\mathbb{R}$	la distribución empírica	normalidad aun con formas no alcanzables por potencias

Las transformaciones sin parámetros no pueden producir fuga; las de potencia y la cuantil estiman parámetros y se ajustan con entrenamiento. Todas cambian la interpretación de los coeficientes: en un modelo para $\log y$, un coeficiente b de x significa que aumentar x en una unidad multiplica la mediana de y por e^b (bajo el modelo de la Proposición 4.4).

Implementación en AWS. Glue DataBrew, acción SKEWNESS con `skewFunction` igual a LOG (base en value), ROOT (índice de la raíz en value) o SQUARE:

```
{
  "RecipeAction": {
    "Operation": "SKEWNESS",
    "Parameters": {
      "sourceColumn": "ingreso",
      "targetColumn": "log_ingreso",
      "skewFunction": "LOG",
      "value": "2.718281828"
    }
  }
}
```

DataBrew no tiene Box-Cox, Yeo-Johnson ni transformación cuantil. **Data Wrangler** no tiene un paso dedicado: se usa *Custom formula*, con expresiones de Spark SQL como `log(x)`, `log1p(x)`, `sqrt(x)` o `cbrt(x)`, o *Custom transform* en pandas o PySpark (la versión de scikit-learn disponible ahí incluye `PowerTransformer`). **SageMaker Processing**: `PowerTransformer(method="box-cox"|"yeo-johnson")`, que estima λ por máxima verosimilitud y estandariza la salida por defecto, y `QuantileTransformer(output_distribution="normal")`.

5. Discretización

Definición 5.1. Dados puntos de corte $-\infty = t_0 < t_1 < \dots < t_{B-1} < t_B = +\infty$, la *discretización* asigna a $u \in \mathbb{R}$ el índice $b \in \{1, \dots, B\}$ tal que $t_{b-1} < u \leq t_b$. Los cortes interiores se calculan con la columna de entrenamiento z : con *ancho igual*, $t_b = z_{(1)} + \frac{b}{B}(z_{(n)} - z_{(1)})$; con *frecuencia igual*, $t_b = Q(b/B)$.

Con cortes por cuantiles, cada intervalo contiene aproximadamente n/B observaciones de entrenamiento; con ancho igual, los intervalos de las colas pueden quedar casi vacíos si la distribución es asimétrica.

Proposición 5.2. *Supóngase que los valores de z son distintos y sea g estrictamente creciente. La discretización por frecuencia igual de $g(z)$ asigna a cada observación de entrenamiento el mismo índice que la de z . La discretización por ancho igual no tiene esta propiedad.*

Demostración. El corte $t_b = Q(b/B)$ es un estadístico de orden $z_{(r)}$ o un punto estrictamente entre $z_{(r)}$ y $z_{(r+1)}$, con r determinado solo por b , B y n ; como los valores son distintos, en ambos casos $\{k : z_k \leq t_b\}$ es el conjunto de índices de rango $\leq r$. Como g es estrictamente creciente, los rangos de $g(z)$ son los de z , y el corte de $g(z)$ separa los mismos índices. Para ancho igual basta un ejemplo: $z = (1, 10, 20, 100)$ con $B = 2$ corta en 50.5 y agrupa $\{1, 10, 20\}$, mientras que $\log_{10} z = (0, 1, 1.30, 2)$ corta en 1 y separa $\{1, 10\}$ de $\{20, 100\}$. $\square$

La discretización pierde información. El caso extremo se cuantifica.

Proposición 5.3 (Costo de dicotomizar). *Sea $X \sim N(0, 1)$, $D = \mathbf{1}\{X > 0\}$ e $Y = \beta X + \varepsilon$ con $\beta \neq 0$ y ε independiente de X con varianza finita. Entonces $\text{Corr}(Y, D) = \sqrt{2/\pi} \text{Corr}(Y, X) \approx 0.80 \text{Corr}(Y, X)$.*

Demostración. $\text{Cov}(X, D) = \mathbb{E}[X \mathbf{1}\{X > 0\}] = \int_0^\infty x \varphi(x) dx = \varphi(0) = 1/\sqrt{2\pi}$, usando $\varphi'(x) = -x\varphi(x)$; $\text{Var } D = 1/4$ y $\text{Var } X = 1$, así que $\text{Corr}(X, D) = \frac{1/\sqrt{2\pi}}{1/2} = \sqrt{2/\pi}$. Como ε es independiente de X , también lo es de D , y $\text{Cov}(Y, D) = \beta \text{Cov}(X, D)$ y $\text{Cov}(Y, X) = \beta$. Entonces, con σ_Y y σ_D las desviaciones estándar de Y y de D , $\text{Corr}(Y, D) = \frac{\beta \text{Cov}(X, D)}{\sigma_Y \sigma_D} = \frac{\beta}{\sigma_Y} \text{Corr}(X, D) = \text{Corr}(Y, X) \text{Corr}(X, D)$, pues $\text{Corr}(Y, X) = \beta/\sigma_Y$ (y $\sigma_X = 1$, así que $\text{Corr}(X, D) = \text{Cov}(X, D)/\sigma_D$). $\square$

Una lectura aproximada en términos de tamaño muestral: para probar asociación con la correlación muestral r , se usa que $\sqrt{n}r$ es aproximadamente $N(0, 1)$ bajo independencia (con momentos cuartos finitos), y bajo la alternativa su centro es aproximadamente $\sqrt{n}\rho$. Mantener $\sqrt{n}\rho$ cuando ρ se reduce por el factor $\sqrt{2/\pi}$ exige multiplicar n por $\pi/2 \approx 1.57$: dicotomizar equivale, en esta aproximación, a descartar un 36 % de la muestra.

Proposición 5.4 (Error de aproximación). *Sea f lipschitziana de constante L en $[t_{b-1}, t_b]$, de ancho $w = t_b - t_{b-1}$, y c el punto medio del intervalo. Entonces $|f(u) - f(c)| \leq Lw/2$ para todo u del intervalo.*

Demostración. $|f(u) - f(c)| \leq L|u - c| \leq Lw/2$, porque ningún punto del intervalo dista más de $w/2$ de su punto medio. $\square$

Si la media de la respuesta es una función f lipschitziana de x , la mejor constante en cada intervalo aproxima f con error a lo sumo $Lw/2$, y la media de f sobre los puntos del intervalo, que es lo que estima un modelo con una constante por intervalo (discretización seguida de *one-hot*, §6.3), con error a lo sumo Lw (porque esa media está entre el mínimo y el máximo de f en el intervalo, que difieren en a lo sumo Lw). El sesgo disminuye con w ; pero con intervalos más estrechos cada constante se estima con menos observaciones y con más varianza. A cambio de la pérdida de información, la discretización acota la influencia de los valores extremos (todos caen en el último intervalo, con independencia de su magnitud), permite a un modelo lineal representar relaciones no lineales mediante funciones escalonadas y es el paso previo del WoE para variables continuas (§7.2). En árboles no aporta: los cortes posibles pasan a ser a lo sumo $B - 1$, un subconjunto de los que el árbol examina sobre la variable original (Lema 1.6).

Implementación en AWS. Glue DataBrew, acción BUCKETIZATION, con cortes explícitos en `splits` (admite `-Infinity` e `Infinity`) o con `percentage` para intervalos de igual frecuencia; BINARIZATION para un único umbral:

```
{
  "Action": {
    "Operation": "BUCKETIZATION",
    "Parameters": {
      "sourceColumn": "peso",
      "targetColumn": "peso_bin",
      "bucketNames": ["Bin1", "Bin2", "Bin3"],
      "splits": ["-Infinity", "2", "20", "Infinity"]
    }
  }
}
```

Data Wrangler no tiene paso de discretización en su lista vigente; se implementa con *Custom formula* (CASE WHEN) o *Custom transform* en PySpark (Bucketizer, QuantileDiscretizer), seguido de *Encode categorical* $\rightarrow$ *One-hot encode*. **SageMaker Processing:** `KBinsDiscretizer(strategy="uniform"|"quantile"|"kmeans", encode="onehot")`.

6. Características categóricas: codificaciones no supervisadas

6.1. Planteamiento

Sea X una característica categórica con niveles $\mathcal{C} = \{c_1, \dots, c_K\}$, los observados en entrenamiento. Una *codificación* es una función $\phi : \mathcal{C} \rightarrow \mathbb{R}^d$; el modelo recibe $\phi(X)$ en lugar de X . Cinco rasgos distinguen las codificaciones: si ϕ es inyectiva (si puede confundir niveles), la dimensión d , la geometría que induce (las distancias $\|\phi(c_i) - \phi(c_j)\|$, que usan los métodos basados en distancias y las redes), si usa la respuesta, y qué valor asigna en inferencia a un nivel que no está en $\mathcal{C}$. Para modelos lineales, el rasgo decisivo es el siguiente.

Definición 6.1. Un *predictor lineal* sobre ϕ es una función $\eta(c) = \beta_0 + \beta^\top \phi(c)$, $(\beta_0, \beta) \in \mathbb{R}^{1+d}$. Su *vector de efectos* es $(\eta(c_1), \dots, \eta(c_K)) \in \mathbb{R}^K$. Sea $\Phi \in \mathbb{R}^{K \times d}$ la matriz cuya fila i es $\phi(c_i)^\top$.

Proposición 6.2 (Efectos representables). *El conjunto de vectores de efectos de los predictores lineales sobre ϕ es el espacio columna de la matriz $[\mathbf{1} \ \Phi] \in \mathbb{R}^{K \times (1+d)}$; su dimensión es $\text{rango}[\mathbf{1} \ \Phi] \leq \min(K, d + 1)$. En particular, todo vector de $\mathbb{R}^K$ es un vector de efectos si y solo si $\text{rango}[\mathbf{1} \ \Phi] = K$.*

Demostración. La coordenada i del vector de efectos es $\beta_0 + \phi(c_i)^\top \beta$, la coordenada i de $[\mathbf{1} \ \Phi](\beta_0, \beta^\top)^\top$. El conjunto de vectores de efectos es, por tanto, la imagen de la aplicación lineal $(\beta_0, \beta) \mapsto [\mathbf{1} \ \Phi](\beta_0, \beta^\top)^\top$, que es el espacio columna; su dimensión es el rango, acotado por el número de filas y de columnas. Un subespacio de $\mathbb{R}^K$ es todo $\mathbb{R}^K$ si y solo si su dimensión es K . $\square$

Si la respuesta media depende de los niveles de forma arbitraria, una codificación con rango menor que K impide a un modelo lineal representarla. Los árboles y las redes no están sujetos a esta proposición, pero sí al orden y a la geometría que ϕ induce.

6.2. Codificación ordinal

Definición 6.3. Dada una biyección $\sigma : \mathcal{C} \rightarrow \{1, \dots, K\}$ (un orden de los niveles), la *codificación ordinal* es $\phi(c) = \sigma(c) \in \mathbb{R}$. El capítulo la llama *label encoding*.

Si la variable es ordinal (tallas $S < M < L < XL$), σ es el orden natural y la codificación lo conserva; si es nominal, σ es arbitraria (`LabelEncoder` de scikit-learn usa el orden alfabético). Con $K \geq 2$, $\text{rango}[\mathbf{1} \ \Phi] = 2$ (las columnas $\mathbf{1}$ y $(1, 2, \dots, K)^\top$ no son proporcionales), así que, por la Proposición 6.2, un predictor lineal solo representa efectos de la forma $\beta_0 + \beta\sigma(c)$: equiespaciados en el orden elegido.

Contraejemplo 6.4. Si los efectos de las tallas S, M, L, XL son $(0, 0, 0, 10)$, ningún (β_0, β) los representa: $\beta_0 + \beta = \beta_0 + 2\beta$ obliga a $\beta = 0$, y entonces el cuarto efecto sería $\beta_0 = 0 \neq 10$. Aun en una variable ordinal, la codificación ordinal restringe la forma del efecto.

Para árboles, el Lema 1.6 dice que un corte sobre $\sigma(X)$ separa los niveles en $\{c : \sigma(c) \leq t\}$ y su complemento: hay $K - 1$ particiones posibles, mientras que el número de particiones de $\mathcal{C}$ en dos conjuntos no vacíos es $(2^K - 2)/2 = 2^{K-1} - 1$ (los 2^K subconjuntos menos $\emptyset$ y $\mathcal{C}$, contando cada partición dos veces). Aislar un nivel de posición t requiere a lo sumo dos cortes: $\sigma \leq t - 1$ y, en el nodo $\{\sigma \geq t\}$, $\sigma \leq t$. Así, un árbol puede representar cualquier función de los niveles con suficientes cortes, pero el orden σ determina cuántos necesita; el Teorema 7.10 identifica un orden con el que basta un corte para la mejor partición en dos.

Implementación en AWS. `Data Wrangler`, *Encode categorical $\rightarrow$ Ordinal encode* (enteros desde 0), con manejo de niveles no vistos *Skip*, *Keep*, *Error* o *Replace with NaN*; paso con parámetros ajustados (*refit*). **Glue DataBrew**, `CATEGORICAL_MAPPING`, que permite fijar σ explícitamente:

```
{
  "Action": {
    "Operation": "CATEGORICAL_MAPPING",
    "Parameters": {
      "sourceColumn": "talla",
      "targetColumn": "talla_ord",
      "categoryMap": "{ \"S\": \"1\", \"M\": \"2\", \"L\": \"3\", \"XL\": \"4\" }",
      "mapType": "NUMERIC",
      "keepOthers": "false",
      "deleteOtherRows": "false",
      "other": "0"
    }
  }
}
```

SageMaker Processing: `OrdinalEncoder(categories=[...])` con el orden explícito.

6.3. Codificación one-hot

Definición 6.5. La codificación *one-hot* es $\phi(c_i) = e_i \in \mathbb{R}^K$, el i -ésimo vector de la base canónica. La *codificación de referencia* elimina la coordenada de un nivel fijado (el de referencia), con $d = K - 1$. Para una muestra, la matriz *one-hot* $D \in \{0, 1\}^{n \times K}$ tiene $D_{ki} = 1$ si y solo si la observación k tiene el nivel c_i .

Proposición 6.6 (Trampa de las variables ficticias). *La matriz $Z = [\mathbf{1} \ D] \in \mathbb{R}^{n \times (K+1)}$ no tiene rango completo por columnas, y $Z^\top Z$ no es invertible.*

Demostración. Cada fila de D tiene exactamente un 1, luego $D\mathbf{1}_K = \mathbf{1}_n$ y $Z(1, -\mathbf{1}_K^\top)^\top = \mathbf{1}_n - D\mathbf{1}_K = 0$ con $(1, -\mathbf{1}_K^\top) \neq 0$. Como $Z^\top Z v = 0$ implica $\|Zv\|^2 = v^\top Z^\top Z v = 0$, el núcleo de $Z^\top Z$ coincide con el de Z , que contiene un vector no nulo. $\square$

Las salidas son omitir un nivel (codificación de referencia), omitir el intercepto o penalizar: con penalización *ridge*, la matriz $Z^\top Z + \lambda I$ con $\lambda > 0$ es definida positiva ($v^\top (Z^\top Z + \lambda I) v = \|Zv\|^2 + \lambda \|v\|^2 > 0$ si $v \neq 0$) y la solución es única.

Proposición 6.7 (Interpretación). *Supóngase que todos los niveles aparecen en la muestra. En la regresión por mínimos cuadrados de y sobre el intercepto y la codificación de referencia (referencia c_K), el valor ajustado de toda observación del nivel c_i es la media $\bar{y}_i$ de las respuestas de ese nivel; los coeficientes son $\hat{\beta}_0 = \bar{y}_K$ y $\hat{\beta}_i = \bar{y}_i - \bar{y}_K$.*

Demostración. El espacio columna de $[\mathbf{1} \ D_{\cdot 1} \cdots D_{\cdot K-1}]$ contiene a $D_{\cdot K} = \mathbf{1} - \sum_{i < K} D_{\cdot i}$, así que coincide con $\text{col}(D)$, el conjunto de vectores que son constantes sobre las observaciones de cada nivel. Por el Lema 1.9, los valores ajustados minimizan $\sum_i \sum_{k: x_k = c_i} (y_k - a_i)^2$ sobre las constantes (a_i) , y cada sumando se minimiza por separado con $a_i = \bar{y}_i$. Las columnas del diseño de referencia son linealmente independientes (si $\beta_0 \mathbf{1} + \sum_{i < K} \beta_i D_{\cdot i} = 0$, evaluando en una observación de c_K se obtiene $\beta_0 = 0$ y luego, en una de cada c_i , $\beta_i = 0$), así que los coeficientes son únicos, y de $\hat{\beta}_0 = \bar{y}_K$ y $\hat{\beta}_0 + \hat{\beta}_i = \bar{y}_i$ se obtienen los valores del enunciado. $\square$

El rango de $[\mathbf{1} \ \Phi]$ es K (las filas $(1, e_i^\top)$ son linealmente independientes), de modo que la codificación es *saturada*: no impone restricción alguna (Proposición 6.2). Su geometría es la opuesta a la ordinal: $\|e_i - e_j\| = \sqrt{2}$ y $\langle e_i, e_j \rangle = 0$ para $i \neq j$, así que no introduce orden ni similitudes artificiales, pero tampoco puede expresar similitudes reales. Su costo es la dimensión y la varianza: bajo el modelo $y_k = \theta_{x_k} + \varepsilon_k$ con errores independientes de varianza σ^2 , el valor ajustado del nivel c_i es $\bar{y}_i$, con varianza σ^2/n_i , que es grande para niveles raros. Un nivel no visto en entrenamiento recibe el vector nulo (`handle_unknown="ignore"`), es decir, la predicción del nivel de referencia o del intercepto. La Proposición 7.4 muestra que la penalización *ridge* sobre *one-hot* reduce esa varianza con el mismo mecanismo que el *target encoding*.

Implementación en AWS. **Data Wrangler**, *Encode categorical $\rightarrow$ One-hot encode*, con la opción *Drop last category* (codificación de referencia, Proposición 6.6), manejo de niveles no vistos y salida como vector disperso o columnas; el paso *Replace rare* (grupo *Handle outliers*) agrupa antes los niveles poco frecuentes en uno solo. **Glue DataBrew**, `ONE_HOT_ENCODING` con `sourceColumn`, que admite como máximo 10 valores distintos. **SageMaker Processing**: `OneHotEncoder(handle_unknown="ignore", drop="first", min_frequency=...)`. Los algoritmos integrados *Linear Learner* y *Factorization Machines* aceptan entrada dispersa en formato `recordIO-protobuf`.

6.4. Codificación binaria

Definición 6.8. Dada una numeración $\sigma : \mathcal{C} \rightarrow \{1, \dots, K\}$, sea d el menor entero con $2^d - 1 \geq K$. La *codificación binaria* asigna a c el vector $\phi(c) \in \{0, 1\}^d$ de los dígitos de $\sigma(c)$ en base 2.

Con d dígitos binarios se escriben los enteros de 0 a $2^d - 1$; como la numeración empieza en 1 (convención de `category_encoders`), se necesita $2^d - 1 \geq K$, es decir, $d = \lceil \log_2(K + 1) \rceil$. Con los 8 colores del capítulo, $d = 4$; con $K = 1000$, $d = 10$. La codificación es inyectiva, porque la expansión binaria de un entero es única: no pierde información.

Proposición 6.9. Si $K \geq 5$, $\text{rango}[\mathbf{1} \ \Phi] \leq d + 1 < K$: un predictor lineal sobre la codificación binaria no representa todos los vectores de efectos.

Demostración. $[\mathbf{1} \ \Phi]$ tiene $d + 1$ columnas. Como $d = \lceil \log_2(K + 1) \rceil < \log_2(K + 1) + 1$, basta ver que $\log_2(K + 1) + 2 \leq K$ para $K \geq 5$: vale en $K = 5$ ($\log_2 6 + 2 \approx 4.58$) y la función $K - \log_2(K + 1)$ es creciente para $K \geq 1$ (su derivada es $1 - 1/((K + 1) \ln 2) > 0$). La conclusión se sigue de la Proposición 6.2. $\square$

La geometría inducida depende de la numeración. Con la numeración por orden de aparición del capítulo (White = 1, ..., Brown = 8), Red (0011) y Pink (0111) difieren en un bit y White (0001) y Brown (1000) en dos; con otra numeración, las distancias serían otras. En árboles, un corte sobre un bit separa los niveles según ese dígito, otra partición fijada por la numeración. La

codificación binaria reduce la dimensión de *one-hot* de K a $\approx \log_2 K$ a cambio de una estructura que no proviene de los datos.

Implementación en AWS. No hay paso integrado en Data Wrangler ni en Data-Brew. En **Data Wrangler**, *Ordinal encode* seguido de una *Custom formula* de Spark SQL por bit, `shiftright(color_idx, j) & 1`, $j = 0, \dots, d - 1$. En **SageMaker Processing**, `category_encoders.BinaryEncoder`, instalado en el contenedor.

6.5. Hashing de características

Qué se representa. El *hashing* se aplica a vectores indexados por *símbolos*. Sea $\mathcal{T}$ el conjunto de símbolos posibles, que puede ser infinito o desconocido de antemano: todas las cadenas de caracteres que podrían aparecer como nivel de una categórica, o todos los tokens que podrían aparecer en un texto (§10). Sea $\mathbb{R}^{(\mathcal{T})}$ el espacio de vectores $x = (x_t)_{t \in \mathcal{T}}$ con solo un número finito de coordenadas no nulas, con producto interno $\langle x, x' \rangle = \sum_t x_t x'_t$ (suma finita). Los dos casos del capítulo son:

- (a) una característica categórica: la observación con nivel c se representa por $x = e_c$, con $x_c = 1$ y $x_t = 0$ para $t \neq c$ (la codificación *one-hot* indexada por todos los niveles posibles);
- (b) un documento: tras tokenizarlo, x_t es el número de veces que el token t aparece en él (la bolsa de palabras de §10.4).

El problema. Usar x directamente requiere un diccionario que asigne a cada símbolo observado en entrenamiento una columna. Su memoria crece con el número de símbolos distintos (millones en filtros de *spam* o en identificadores de usuario), y un símbolo que aparece por primera vez en producción no tiene columna. Weinberger et al. (2009) propusieron fijar de antemano el número de columnas.

Definición 6.10 (*Hashing* de características). Sean $m \in \mathbb{N}$, $h : \mathcal{T} \rightarrow \{1, \dots, m\}$ y $\xi : \mathcal{T} \rightarrow \{-1, +1\}$ dos funciones. La representación con *hashing* de $x \in \mathbb{R}^{(\mathcal{T})}$ es $\phi(x) \in \mathbb{R}^m$ con

$$\phi_j(x) = \sum_{t: h(t)=j} \xi(t) x_t, \quad j = 1, \dots, m,$$

suma finita porque x tiene finitas coordenadas no nulas. En el caso (a), $\phi(e_c) = \xi(c) e_{h(c)}$: un vector con un solo ± 1 en la posición $h(c)$. En el caso (b), la coordenada j suma, con signo, los conteos de los tokens que h envía a j .

Dos símbolos $t \neq u$ *colisionan* si $h(t) = h(u)$. En la práctica, h y ξ se calculan con una función hash de cadenas: scikit-learn aplica MurmurHash3 con semilla fija a la cadena del símbolo, toma $h(t)$ como el valor absoluto del resultado módulo m y $\xi(t)$ como su signo. Son funciones deterministas: el mismo símbolo produce siempre la misma columna y el mismo signo, sin necesidad de guardar nada. Para analizarlas se usa el modelo siguiente, que describe una elección aleatoria de la función (por ejemplo, de su semilla), con x fijo.

Modelo de análisis. (H1) Para símbolos distintos, los valores $h(t)$ son independientes y uniformes en $\{1, \dots, m\}$. (H2) Los valores $\xi(t)$ son independientes y uniformes en $\{-1, +1\}$. (H3) h y ξ son independientes. (En scikit-learn h y ξ provienen del mismo valor hash, así que (H3) es una idealización.)

Proposición 6.11 (Insensatez del producto interno). *Bajo (H2) y (H3), para todo par $x, x' \in \mathbb{R}^{(\mathcal{T})}$, $\mathbb{E}[\langle \phi(x), \phi(x') \rangle] = \langle x, x' \rangle$.*

Demostración. Sea S el conjunto finito de símbolos con $x_t \neq 0$ o $x'_t \neq 0$. Entonces $\langle \phi(x), \phi(x') \rangle = \sum_{j=1}^m \sum_{t,u \in S} \mathbf{1}\{h(t) = j\} \mathbf{1}\{h(u) = j\} \xi(t) \xi(u) x_t x'_u = \sum_{t,u \in S} \mathbf{1}\{h(t) = h(u)\} \xi(t) \xi(u) x_t x'_u$. Por (H3), la esperanza de cada sumando es $\mathbb{P}(h(t) = h(u)) \mathbb{E}[\xi(t) \xi(u)] x_t x'_u$. Si $t \neq u$, por (H2) $\mathbb{E}[\xi(t) \xi(u)] = \mathbb{E}[\xi(t)] \mathbb{E}[\xi(u)] = 0$; si $t = u$, $\xi(t)^2 = 1$ y $\mathbb{P}(h(t) = h(t)) = 1$. Queda $\sum_{t \in S} x_t x'_t$. $\square$

En el caso (a) con $c \neq c'$, $\langle e_c, e_{c'} \rangle = 0$: en promedio sobre la elección de la función, los niveles distintos siguen siendo ortogonales, como en *one-hot*. El signo es la causa: si $\xi \equiv 1$, el sumando de un par $t \neq u$ tiene esperanza $\mathbb{P}(h(t) = h(u)) x_t x'_u = x_t x'_u / m$ bajo (H1), y el producto interno queda sesgado en $\sum_{t \neq u} x_t x'_u / m$, positivo para conteos: las colisiones siempre sumarían. Con signo, se cancelan en promedio; por eso `FeatureHasher` usa `alternate_sign=True` y en la salida del capítulo aparecen valores ± 1 .

Proposición 6.12 (Número de colisiones). *Bajo (H1), entre K símbolos distintos el número esperado de pares que colisionan es $\binom{K}{2}/m$, y la probabilidad de que no haya ninguna colisión es $\prod_{i=1}^{K-1} (1 - i/m) \leq \exp(-K(K-1)/(2m))$. Un símbolo dado colisiona con algún otro con probabilidad $1 - (1 - 1/m)^{K-1}$.*

Demostración. Para un par, $\mathbb{P}(h(t) = h(u)) = \sum_{j=1}^m \mathbb{P}(h(t) = j) \mathbb{P}(h(u) = j) = m \cdot m^{-2} = 1/m$; la esperanza del número de pares en colisión es la suma sobre los $\binom{K}{2}$ pares de sus indicadoras. Para la segunda afirmación, se asignan los símbolos en orden: dado que los i primeros ocupan i valores distintos, el siguiente evita esos valores con probabilidad $1 - i/m$ por la independencia de (H1); el producto de estas probabilidades condicionales es la probabilidad de no colisión, y la cota usa $1 - v \leq e^{-v}$ y $\sum_{i=1}^{K-1} i = K(K-1)/2$. La tercera: el símbolo evita el valor de cada uno de los otros $K-1$ independientemente con probabilidad $1 - 1/m$. $\square$

Ejemplo 6.13. En el ejemplo del capítulo hay $K = 8$ colores y $m = 3$. El número esperado de pares en colisión es $28/3 \approx 9.3$. En la salida real, la columna `feature_1` recibe White, Black y Blue (3 pares), la columna `feature_2` recibe Green, Yellow, Pink y Brown (6 pares) y Red queda sola en `feature_0`: 9 pares. El signo distingue a Black (+1) de White y Blue (−1), y a Yellow (+1) de Green, Pink y Brown (−1); pero White y Blue quedan con el mismo vector, y también Green, Pink y Brown. Por el principio del palomar, con $m = 3$ columnas y dos signos hay a lo sumo 6 vectores distintos para 8 colores, así que alguna coincidencia era inevitable.

Si K es del orden de m , la tercera fórmula da $1 - (1 - 1/m)^{K-1} \approx 1 - e^{-1} \approx 0.63$: la mayoría de los símbolos comparte columna. Por eso se usan valores de m del orden de 2^{18} a 2^{20} y se acepta que colisionen sobre todo los símbolos raros.

Contraste. Frente a *one-hot*, el *hashing* fija la dimensión, no guarda diccionario y procesa símbolos nuevos sin reentrenar; a cambio no es inyectivo ni invertible (una columna no indica qué símbolos la forman) y m es un hiperparámetro. Frente a la codificación binaria, ambas reducen dimensión, pero la binaria es inyectiva y el *hashing* no; el *hashing* conserva en promedio los productos internos de *one-hot* (Proposición 6.11) y la binaria introduce similitudes fijadas por la numeración. Como no tiene parámetros que ajustar, el *hashing* no puede producir fuga, siempre que h , ξ y m sean los mismos en entrenamiento y en producción.

Variante: codificación por similitud con *min-hash*

Cuando los niveles son cadenas con variantes de escritura («juan perez», «juan peres»), *one-hot* y *hashing* las tratan como símbolos ajenos. La codificación por similitud (Cerdeja y Varoquaux, 2022) representa cada nivel por sus fragmentos de tres caracteres.

Definición 6.14. Sea $\mathcal{A}$ un alfabeto finito y $U = \mathcal{A}^3$ el conjunto de cadenas de longitud 3. Para una cadena c de longitud ≥ 3 , sea $S(c) \subset U$ el conjunto de sus subcadenas de tres caracteres consecutivos (sus *3-gramas*). La *similitud de Jaccard* de dos conjuntos no vacíos es

$J(A, B) = |A \cap B|/|A \cup B|$. Para una biyección $\pi : U \rightarrow \{1, \dots, |U|\}$ (una permutación de U), el *min-hash* de $S \subset U$ no vacío es $h_\pi(S) = \min_{g \in S} \pi(g)$.

Por ejemplo, «juan perez» y «juan peres» tienen 7 3-gramas en común de 9 distintos en total, $J = 7/9 \approx 0.78$; «juan perez» y «maria lopez» no comparten ninguno, $J = 0$.

Proposición 6.15 (Broder, 1997). *Si π es una permutación aleatoria uniforme de U , entonces $\mathbb{P}(h_\pi(A) = h_\pi(B)) = J(A, B)$ para $A, B \subset U$ no vacíos.*

Demostración. Sea g^* el elemento de $A \cup B$ con menor valor de π . Como π es uniforme, todos los elementos de $A \cup B$ tienen la misma probabilidad de ser g^* , así que $\mathbb{P}(g^* \in A \cap B) = |A \cap B|/|A \cup B|$. Si $g^* \in A \cap B$, es el mínimo de ambos conjuntos y $h_\pi(A) = h_\pi(B) = \pi(g^*)$. Si $g^* \in A \setminus B$, entonces $h_\pi(A) = \pi(g^*)$, mientras que $h_\pi(B) > \pi(g^*)$ porque todo elemento de B está en $A \cup B$ y es distinto de g^* ; análogamente si $g^* \in B \setminus A$. Luego $h_\pi(A) = h_\pi(B)$ si y solo si $g^* \in A \cap B$. $\square$

La codificación con d permutaciones independientes $\pi_1, \dots, \pi_d$ es

$$\phi(c) = (h_{\pi_1}(S(c)), \dots, h_{\pi_d}(S(c))).$$

La fracción de coordenadas en que coinciden $\phi(c)$ y $\phi(c')$ es un promedio de d indicadoras independientes de media $J(S(c), S(c'))$, luego estima esa similitud sin sesgo y con varianza $J(1 - J)/d$. Niveles con muchos 3-gramas comunes reciben vectores que coinciden en muchas coordenadas: la geometría refleja similitud de escritura, lo contrario de las colisiones arbitrarias del *hashing*. Las implementaciones sustituyen las permutaciones aleatorias por funciones hash, que las aproximan.

Implementación en AWS. Data Wrangler: *Encode categorical* $\rightarrow$ *Similarity encode* implementa la codificación por similitud (3-gramas y *min-hash*, dimensión $d = 30$ por defecto, salida como vector o columnas). El *hashing* está en *Featurize text* $\rightarrow$ *Vectorize* con el vectorizador *Hashing* (número de columnas m configurable), pensado para el caso (b); para una categórica (caso (a)) se usa *Custom transform* con *FeatureHasher* de PySpark o de scikit-learn. **SageMaker Processing:** `FeatureHasher(n_features=2**20, input_type="string")`.

7. Características categóricas: codificaciones supervisadas

Con cardinalidad alta, las codificaciones anteriores fallan de algún modo: *one-hot* estima un efecto por nivel con varianza σ^2/n_i ; la ordinal impone un orden arbitrario; la binaria y el *hashing* introducen estructura ajena a los datos. Para pérdida cuadrática, la función del nivel que mejor predice Y es $\mathbb{E}[Y \mid X = c_i]$ (por el argumento de la demostración de la Proposición 8.4, con la historia reemplazada por X), de modo que codificar cada nivel por una estimación de esa cantidad resume en un número lo que el nivel dice sobre Y . Es la idea del *target encoding*; el *weight of evidence* es su versión en la escala de log-momios.

7.1. Target encoding

Notación. En la muestra de entrenamiento, para el nivel c_i : n_i es su número de observaciones, $L_i = \{k : x_k = c_i\}$ su conjunto de índices y $\bar{y}_i = n_i^{-1} \sum_{k \in L_i} y_k$ su media de la respuesta; $\bar{y}$ es la media global e $i(k)$ el índice del nivel de la observación k . Si $y \in \{0, 1\}$, $\bar{y}_i$ es la proporción de eventos del nivel.

Definición 7.1 (*Target encoding*). Para $m \geq 0$, la codificación del nivel c_i es

$$S_i = \lambda_i \bar{y}_i + (1 - \lambda_i) \bar{y}, \quad \lambda_i = \frac{n_i}{n_i + m}.$$

Con $m = 0$ es la media del nivel sin suavizar. A un nivel no visto en entrenamiento ($n_i = 0$, $\lambda_i = 0$) le corresponde $\bar{y}$. Si la respuesta tiene C clases, se calcula una columna por clase j con y reemplazada por $\mathbf{1}\{y = j\}$.

El suavizado responde al problema señalado por Micci-Barreca (2001): un nivel observado una vez con $y = 1$ tendría $\bar{y}_i = 1$, una estimación basada en un solo dato. El peso λ_i interpola entre la media del nivel, fiable si n_i es grande, y la global, preferible si n_i es pequeño; m es el número de observaciones con el que ambas pesan lo mismo. Los dos resultados siguientes muestran que esta forma es la media posterior de modelos jerárquicos estándar.

Proposición 7.2 (Respuesta binaria). *Supóngase que, dado θ_i , las respuestas del nivel c_i son Bernoulli(θ_i) independientes, y que a priori $\theta_i \sim \text{Beta}(a, b)$ con $a, b > 0$. Entonces*

$$\mathbb{E}[\theta_i \mid (y_k)_{k \in L_i}] = \frac{a + n_i \bar{y}_i}{a + b + n_i} = \lambda_i \bar{y}_i + (1 - \lambda_i) \frac{a}{a + b}, \quad \lambda_i = \frac{n_i}{n_i + a + b}.$$

Demostración. Con $s = n_i \bar{y}_i$ eventos, la verosimilitud es $\theta_i^s (1 - \theta_i)^{n_i - s}$ y la densidad a priori es proporcional a $\theta_i^{a-1} (1 - \theta_i)^{b-1}$; la posterior es proporcional a su producto, $\theta_i^{a+s-1} (1 - \theta_i)^{b+n_i-s-1}$, que es la densidad $\text{Beta}(a + s, b + n_i - s)$, de media $(a + s)/(a + b + n_i)$. Escribiendo $a + s = (a + b) \frac{a}{a+b} + n_i \bar{y}_i$ y dividiendo por $a + b + n_i$ se obtiene la segunda forma. $\square$

La codificación es la media posterior con media a priori $a/(a+b)$ (estimada por $\bar{y}$) y $m = a+b$, que actúa como un número de observaciones previas.

Proposición 7.3 (Respuesta continua). *Supóngase que, dado θ_i , las respuestas del nivel son $N(\theta_i, \sigma^2)$ independientes, y que $\theta_i \sim N(\mu, \tau^2)$, con $\mu, \sigma^2 > 0$ y $\tau^2 > 0$ conocidos. Entonces la distribución de θ_i dada la muestra del nivel es $N(S_i, v_i)$ con*

$$S_i = \lambda_i \bar{y}_i + (1 - \lambda_i) \mu, \quad \lambda_i = \frac{n_i}{n_i + \sigma^2 / \tau^2}, \quad v_i = \left(\frac{1}{\tau^2} + \frac{n_i}{\sigma^2} \right)^{-1}.$$

Además $\mathbb{E}[(S_i - \theta_i)^2] = v_i < \sigma^2 / n_i = \mathbb{E}[(\bar{y}_i - \theta_i)^2]$.

Demostración. La verosimilitud de θ_i depende de los datos solo a través de $\bar{y}_i$, que dado θ_i es $N(\theta_i, \sigma^2 / n_i)$. El logaritmo de la densidad posterior es, salvo constantes, $-\frac{n_i}{2\sigma^2} (\bar{y}_i - \theta_i)^2 - \frac{1}{2\tau^2} (\theta_i - \mu)^2$, un polinomio de grado 2 en θ_i con coeficiente cuadrático $-\frac{1}{2} \left(\frac{n_i}{\sigma^2} + \frac{1}{\tau^2} \right) = -\frac{1}{2v_i}$ y coeficiente lineal $\frac{n_i \bar{y}_i}{\sigma^2} + \frac{\mu}{\tau^2}$. Completando el cuadrado, la posterior es normal con varianza v_i y media $v_i \left(\frac{n_i \bar{y}_i}{\sigma^2} + \frac{\mu}{\tau^2} \right) = \frac{n_i \tau^2 \bar{y}_i + \sigma^2 \mu}{n_i \tau^2 + \sigma^2} = \lambda_i \bar{y}_i + (1 - \lambda_i) \mu$. Para los errores: condicionando en los datos, $\mathbb{E}[(S_i - \theta_i)^2 \mid \text{datos}] = v_i$ (la varianza posterior), que no depende de los datos; condicionando en θ_i , $\mathbb{E}[(\bar{y}_i - \theta_i)^2 \mid \theta_i] = \sigma^2 / n_i$. La desigualdad $v_i < \sigma^2 / n_i$ equivale a $\frac{1}{\tau^2} + \frac{n_i}{\sigma^2} > \frac{n_i}{\sigma^2}$. $\square$

La proposición identifica $m = \sigma^2 / \tau^2$: la dispersión dentro de los niveles frente a la dispersión entre sus medias. En la práctica μ , σ^2 y τ^2 se estiman con los datos (Bayes empírico). `TargetEncoder(smooth="auto")` de scikit-learn usa, según su documentación, $m_i = \sigma_i^2 / \tau^2$, con σ_i^2 la varianza de y dentro del nivel y τ^2 la varianza de y en toda la muestra; en el modelo, τ^2 es la varianza entre medias de nivel, y por la ley de la varianza total la de y es $\sigma^2 + \tau^2 > \tau^2$, de modo que esa elección suaviza menos que la regla del modelo.

Proposición 7.4 (Ridge sobre one-hot es un suavizado). *Supóngase que todos los niveles aparecen y sea $\lambda > 0$. La función $F(\beta_0, \beta) = \sum_k (y_k - \beta_0 - \beta_{i(k)})^2 + \lambda \sum_{i=1}^K \beta_i^2$ tiene un único minimizador, y en él el valor ajustado para el nivel c_i es $\hat{\beta}_0 + \lambda_i (\bar{y}_i - \hat{\beta}_0)$ con $\lambda_i = n_i / (n_i + \lambda)$.*

Demostración. F es cuadrática con hessiana $2(Z^\top Z + \text{diag}(0, \lambda I_K))$, con $Z = [\mathbf{1} \ D]$. Si $v = (v_0, v_1, \dots, v_K)$ anula la forma cuadrática, $\|Zv\|^2 + \lambda \sum_{i \geq 1} v_i^2 = 0$, así que $v_i = 0$ para $i \geq 1$ y $Zv = v_0 \mathbf{1} = 0$, es decir, $v = 0$: la hessiana es definida positiva, F es estrictamente convexa y su único minimizador es el punto donde se anula el gradiente. La derivada respecto de β_i es $-2 \sum_{k \in L_i} (y_k - \beta_0 - \beta_i) + 2\lambda \beta_i$; igualada a cero, $(n_i + \lambda) \beta_i = n_i (\bar{y}_i - \beta_0)$, de donde $\hat{\beta}_i = \lambda_i (\bar{y}_i - \hat{\beta}_0)$, y el valor ajustado del nivel es $\hat{\beta}_0 + \hat{\beta}_i$. $\square$

La penalización *ridge* sobre K columnas *one-hot* y el *target encoding* en una columna producen la misma contracción, con $m = \lambda$ y $\hat{\beta}_0$ en el papel de la media a priori. La diferencia práctica es que el *target encoding* reduce la dimensión a 1 antes de modelar, y la columna resultante puede usarse con cualquier algoritmo.

7.1.1. Fuga de información dentro del entrenamiento

La Proposición 1.3 exige no usar la muestra de prueba para ajustar; el resultado siguiente muestra que, además, usar la propia observación en su codificación distorsiona el entrenamiento.

Proposición 7.5 (Codificación dentro de la muestra). *Supóngase que $y_k = \theta_{i(k)} + \varepsilon_k$, con constantes $\theta_1, \dots, \theta_K$ y errores $\varepsilon_1, \dots, \varepsilon_n$ independientes, de media 0 y varianza σ^2 , y considérense fijos los niveles de las observaciones. Si $S_{i(k)}$ se calcula con toda la muestra de entrenamiento, entonces, con $i = i(k)$,*

$$\text{Cov}(S_i, \varepsilon_k) = \lambda_i \frac{\sigma^2}{n_i} + (1 - \lambda_i) \frac{\sigma^2}{n} > 0,$$

mientras que para una observación nueva, cuyo error es independiente de los de entrenamiento, la covarianza correspondiente es 0.

Demostración. $\bar{y}_i = \theta_i + n_i^{-1} \sum_{l \in L_i} \varepsilon_l$ y $\bar{y} = n^{-1} \sum_l \theta_{i(l)} + n^{-1} \sum_l \varepsilon_l$. Por la independencia de los errores, $\text{Cov}(\varepsilon_l, \varepsilon_k)$ vale σ^2 si $l = k$ y 0 si no, de modo que $\text{Cov}(\bar{y}_i, \varepsilon_k) = \sigma^2/n_i$ (pues $k \in L_i$) y $\text{Cov}(\bar{y}, \varepsilon_k) = \sigma^2/n$; S_i es la combinación lineal de $\bar{y}_i$ y $\bar{y}$ con coeficientes λ_i y $1 - \lambda_i$. Para una observación nueva, su error no aparece en $\bar{y}_i$ ni en $\bar{y}$ y es independiente de todos los que aparecen. $\square$

En entrenamiento, la característica contiene parte del error de la propia observación y el modelo aprende a apoyarse en ella; en producción esa parte no existe. Con $n_i = 1$ y $m = 0$, $S_{i(k)} = y_k$: la característica es la respuesta.

Contraejemplo 7.6 (*Leave-one-out*). Codificar cada observación con la media de las demás de su nivel, $S_{-k} = (T_i - y_k)/(n_i - 1)$ con $T_i = \sum_{l \in L_i} y_l$ y $n_i \geq 2$, elimina la covarianza anterior, porque ε_k ya no interviene. Pero dentro del nivel S_{-k} es una función afín estrictamente decreciente de y_k , con los mismos T_i y n_i para todas las observaciones del nivel. En un nivel con respuestas $(1, 1, 0, 0)$, las observaciones con $y = 1$ reciben $S_{-k} = 1/3$ y las de $y = 0$, $S_{-k} = 2/3$: el corte $S_{-k} \leq 1/2$ las clasifica todas correctamente en entrenamiento, mientras que en producción toda observación del nivel recibe la media del nivel, $1/2$, y el corte no aporta nada.

Proposición 7.7 (Codificación cruzada). *Particiónese el conjunto de índices de entrenamiento en V pliegues $F_1, \dots, F_V$ y, para $k \in F_v$, sea S_k^{cf} la codificación del nivel de k calculada, con la fórmula de la definición, usando solo las observaciones fuera de F_v . Bajo el modelo de la Proposición 7.5 y con los pliegues fijos: (a) S_k^{cf} es independiente de ε_k ; (b) dos observaciones del mismo pliegue y el mismo nivel reciben el mismo valor.*

Demostración. (a) S_k^{cf} es función de las constantes θ_i y de los errores $\{\varepsilon_l : l \notin F_v\}$, independientes de ε_k porque $k \in F_v$. (b) Si $k, k' \in F_v$ tienen el mismo nivel, ambas codificaciones se calculan con las mismas observaciones y la misma fórmula. $\square$

La parte (b) es la que falla en *leave-one-out*: dentro de un pliegue la codificación de un nivel es constante y no puede ordenar las respuestas de ese nivel. Este esquema es el de `TargetEncoder.fit_transform` de `scikit-learn` ($V = 5$ por defecto); por eso `fit(X,y).transform(X)` no coincide con `fit_transform(X,y)`. En validación, prueba y producción se usa `transform`, con la codificación calculada sobre todo el entrenamiento, independiente del error de la observación nueva (Proposición 7.5).

Observación 7.8 (Estadísticos ordenados de CatBoost). Prokhorenkova et al. (2018) toman una permutación aleatoria ς de los índices de entrenamiento, una constante μ_0 fijada de antemano, y codifican la observación k con

$$S_k^{\text{ord}} = \frac{\sum_{l: \varsigma(l) < \varsigma(k), i(l)=i(k)} y_l + m \mu_0}{\#\{l : \varsigma(l) < \varsigma(k), i(l) = i(k)\} + m}.$$

S_k^{ord} solo depende de respuestas de observaciones anteriores a k en la permutación, y el argumento de la Proposición 7.7(a) da su independencia de ε_k . A cambio, las primeras observaciones de la permutación se codifican con pocos datos; CatBoost usa varias permutaciones para reducir esa varianza.

7.1.2. El orden de las medias y los árboles

El resultado siguiente, de Fisher (1958) para regresión y de Breiman et al. (1984) para clasificación binaria, explica por qué el *target encoding* es adecuado para árboles.

Lema 7.9 (Descomposición de la suma de cuadrados). *Sea N un conjunto de índices particionado en N_A y N_B no vacíos, de tamaños n_A, n_B y medias $\bar{y}_A, \bar{y}_B$, y sea $\bar{y}_N$ la media en N . Entonces $\text{SC}(N) = \text{SC}(N_A) + \text{SC}(N_B) + n_A(\bar{y}_A - \bar{y}_N)^2 + n_B(\bar{y}_B - \bar{y}_N)^2$.*

Demostración. Para $k \in N_A$, $y_k - \bar{y}_N = (y_k - \bar{y}_A) + (\bar{y}_A - \bar{y}_N)$; al elevar al cuadrado y sumar sobre N_A , el término cruzado es $2(\bar{y}_A - \bar{y}_N) \sum_{k \in N_A} (y_k - \bar{y}_A) = 0$. Se obtiene $\text{SC}(N_A) + n_A(\bar{y}_A - \bar{y}_N)^2$; lo mismo para N_B , y se suma. $\square$

Así, la reducción Δ de la Definición 1.5 para una partición de N en N_A y N_B es $\mathcal{B} = n_A(\bar{y}_A - \bar{y}_N)^2 + n_B(\bar{y}_B - \bar{y}_N)^2$.

Teorema 7.10. *Sea N un nodo en el que aparecen los niveles $c_1, \dots, c_K$, con $n_i > 0$ observaciones y medias $\bar{y}_i$ en N . Para $A \subset \{1, \dots, K\}$ no vacío y distinto del total, sea N_A el conjunto de observaciones de N con nivel en A , $N_B = N \setminus N_A$, y $\mathcal{B}(A)$ la cantidad anterior. Existe un maximizador de $\mathcal{B}$ formado por los primeros niveles de una ordenación por $\bar{y}_i$ no decreciente. En consecuencia, examinando los $K - 1$ cortes de la codificación $\bar{y}_i$ (calculada en el nodo) se encuentra la mejor de las $2^{K-1} - 1$ particiones.*

Demostración. Si todas las $\bar{y}_i$ son iguales, $\bar{y}_A = \bar{y}_B = \bar{y}_N$ para todo A y todo A es maximizador. En otro caso, sea A un maximizador. El conjunto A_0 de niveles con media mínima cumple $\bar{y}_{A_0} < \bar{y}_N$, luego $\mathcal{B}(A_0) > 0$; por tanto $\mathcal{B}(A) > 0$ y $\bar{y}_A \neq \bar{y}_B$ (si fueran iguales, ambas coincidirían con $\bar{y}_N$ y $\mathcal{B}(A) = 0$). Intercambiando los nombres si hace falta, $\bar{y}_A < \bar{y}_B$. Sean $\delta_i = \bar{y}_i - \bar{y}_N$, $\alpha = \bar{y}_A - \bar{y}_N$, $\beta = \bar{y}_B - \bar{y}_N$ (luego $\alpha < \beta$), $s_A = n_A \alpha$, $s_B = n_B \beta$ y $G(s, w) = s^2/w$ para $w > 0$, de modo que $\mathcal{B}(A) = G(s_A, n_A) + G(s_B, n_B)$.

Conexidad de G . Su hessiana es $\begin{pmatrix} 2/w & -2s/w^2 \\ -2s/w^2 & 2s^2/w^3 \end{pmatrix}$, de traza positiva y determinante $4s^2/w^4 - 4s^2/w^4 = 0$; sus autovalores son 0 y un número positivo, y G es convexa en $\{w > 0\}$.

Traslado parcial de un nivel. Para $i \in A$ y $t \in [0, 1]$, sea $\Psi(t) = G(s_A - tn_i \delta_i, n_A - tn_i) + G(s_B + tn_i \delta_i, n_B + tn_i)$. Como el nivel i aporta n_i observaciones y $n_i \delta_i$ a la suma $s_A = \sum_{k \in N_A} (y_k - \bar{y}_N)$, $\Psi(1)$ es el valor de $\mathcal{B}$ con el nivel i trasladado a B , es decir, $\mathcal{B}(A \setminus \{i\})$ (o 0 si $A = \{i\}$, por la continuidad que se ve enseguida), y $\Psi(0) = \mathcal{B}(A)$. Cada sumando es G compuesta con una función afín de t , así que Ψ es convexa en $[0, 1]$; si $A = \{i\}$, el primer sumando vale $(1 - t)n_i \delta_i^2$ para $t < 1$ y tiende a 0, y definiendo $\Psi(1)$ por continuidad la convexidad se conserva en $[0, 1]$. La maximalidad de A da $\Psi(1) \leq \Psi(0)$, y por convexidad la derivada por la derecha en 0 cumple $\Psi'(0^+) \leq \Psi(1) - \Psi(0) \leq 0$.

Cálculo. $\partial_s G = 2s/w$ y $\partial_w G = -s^2/w^2$ valen 2α y $-\alpha^2$ en (s_A, n_A) , y 2β y $-\beta^2$ en (s_B, n_B) . Por la regla de la cadena,

$$\Psi'(0^+) = n_i(-2\alpha\delta_i + \alpha^2 + 2\beta\delta_i - \beta^2) = n_i(\beta - \alpha)(2\delta_i - \alpha - \beta).$$

Como $\beta > \alpha$, $\Psi'(0^+) \leq 0$ equivale a $\bar{y}_i \leq \tau$, con $\tau = \bar{y}_N + \frac{\alpha+\beta}{2} = \frac{\bar{y}_A + \bar{y}_B}{2}$. El mismo cálculo trasladando un nivel $j \in B$ hacia A da $n_j(\beta - \alpha)(\alpha + \beta - 2\delta_j) \leq 0$, es decir, $\bar{y}_j \geq \tau$.

Conclusión. Los niveles de A tienen media $\leq \tau$ y los de B , media $\geq \tau$. Ordenando por $\bar{y}_i$ y colocando, entre los empatados en τ , primero los de A , el conjunto A ocupa las primeras posiciones. Si $y \in \{0, 1\}$, $SC(N) = |N|\hat{p}(1 - \hat{p})$ con $\hat{p}$ la proporción de unos en N , que es $|N|/2$ veces el índice de Gini $2\hat{p}(1 - \hat{p})$; el criterio de Gini es entonces proporcional al de suma de cuadrados, y el resultado vale igual. $\square$

Con una codificación ordinal en un orden arbitrario, un corte examina $K - 1$ de las $2^{K-1} - 1$ particiones y en general no la mejor; con el orden de las medias, esas $K - 1$ contienen a la mejor. LightGBM usa esta idea para sus cortes categóricos. El teorema se refiere a las medias *del nodo*; una columna codificada una vez con toda la muestra las da exactamente solo en la raíz.

Ventajas y desventajas. A favor: dimensión 1 para cualquier K ; contracción óptima bajo los modelos jerárquicos (Proposiciones 7.2 y 7.3); valor definido para niveles nuevos; orden óptimo para cortes de árboles (Teorema 7.10). En contra: usa la respuesta, así que requiere codificación cruzada en entrenamiento y ajuste sin la muestra de prueba (Proposiciones 1.3 y 7.5); un predictor lineal sobre S solo representa efectos $\beta_0 + \beta S_i$ (Proposición 6.2 con $d = 1$); no capta interacciones del nivel con otras variables; y la tabla de codificación es un artefacto que debe versionarse y aplicarse igual en producción.

Implementación en AWS. Ni Data Wrangler (Canvas) ni Glue DataBrew incluyen *target encoding* entre sus transformaciones (documentación revisada el 24 de septiembre de 2026). Un *custom transform* de Data Wrangler que calcule medias por nivel lo haría con todos los datos del flujo, sin pliegues, en la situación de la Proposición 7.5. Tres vías respetan los resultados anteriores.

1. **SageMaker Processing** con un script que use **TargetEncoder** (scikit-learn ≥ 1.3 ; se verifica la versión del contenedor o se instala):

```
from sklearn.preprocessing import TargetEncoder
import joblib, pandas as pd
tr = pd.read_parquet("/opt/ml/processing/train/")
te = pd.read_parquet("/opt/ml/processing/test/")
enc = TargetEncoder(smooth="auto", cv=5, target_type="binary")
# entrenamiento: codificación cruzada
tr["cp_te"] = enc.fit_transform(tr[["codigo_postal"]], tr["y"]).ravel()
# prueba: tabla calculada con todo el entrenamiento
te["cp_te"] = enc.transform(te[["codigo_postal"]]).ravel()
joblib.dump(enc, "/opt/ml/processing/model/target_encoder.joblib")
```

El codificador guardado en S3 se carga en el contenedor de inferencia (por ejemplo, el primero de un *inference pipeline*).

2. **Algoritmo integrado CatBoost** de SageMaker, que aplica internamente los estadísticos ordenados. Las columnas categóricas se declaran en **categorical_index.json**, junto a los datos de entrenamiento (CSV sin encabezado con la respuesta en la primera columna; el canal de entrenamiento debe apuntar a un prefijo de S3):

```
{"cat_index_list": [1, 3, 5]}
```

3. **AWS Glue** (PySpark) para volúmenes grandes: se asigna un pliegue a cada fila, se calculan con **groupBy** los conteos y sumas por nivel y pliegue, y cada fila se codifica con las sumas de los demás pliegues. La tabla final (nivel, n_i , S_i) puede publicarse en un *feature group* de **Feature Store** con el nivel como identificador, para consultarla desde el *online store* en inferencia.

7.2. Weight of evidence e information value

Turing y Good (1950) llamaron peso de la evidencia al logaritmo del cociente de verosimilitudes que una observación aporta a favor de una hipótesis frente a otra. La banca lo adoptó para construir *scorecards* de crédito, que debían ser modelos aditivos, con una contribución por variable legible por un analista; la forma que sigue es la de Siddiqi (2006).

Notación. La respuesta es binaria: $Y = 1$ denota el evento («malo») e $Y = 0$ su ausencia («bueno»). X es categórica o una variable continua discretizada (§5). En entrenamiento, $n_i^{(0)}$ y $n_i^{(1)}$ son los números de buenos y de malos con nivel c_i , y $N^{(0)} = \sum_i n_i^{(0)}$, $N^{(1)} = \sum_i n_i^{(1)}$.

Definición 7.11. Supóngase $n_i^{(0)} > 0$ y $n_i^{(1)} > 0$ para todo i . Con $g_i = n_i^{(0)}/N^{(0)}$ y $b_i = n_i^{(1)}/N^{(1)}$ (las distribuciones de X entre buenos y entre malos),

$$\text{WoE}_i = \log \frac{g_i}{b_i}, \quad \text{IV} = \sum_{i=1}^K (g_i - b_i) \text{WoE}_i.$$

Algunos textos usan el cociente inverso, lo que cambia el signo del WoE y no el IV. La hipótesis de conteos positivos es necesaria: un nivel sin malos tendría $\text{WoE} = +\infty$. Las soluciones habituales son sumar 0.5 a cada conteo o fusionar el nivel con otro; ambas son formas de suavizado, comparables con m en el *target encoding*.

Proposición 7.12 (WoE y log-momios). *En la población, sean $\pi = \mathbb{P}(Y = 1) \in (0, 1)$, $G_i = \mathbb{P}(X = c_i \mid Y = 0) > 0$, $B_i = \mathbb{P}(X = c_i \mid Y = 1) > 0$ y $W_i = \log(G_i/B_i)$. Entonces $\text{logit } \mathbb{P}(Y = 1 \mid X = c_i) = \text{logit } \pi - W_i$, donde $\text{logit } u = \log(u/(1-u))$. En la muestra, con $\hat{\pi} = N^{(1)}/(N^{(0)} + N^{(1)})$ y $\bar{y}_i = n_i^{(1)}/(n_i^{(0)} + n_i^{(1)})$, $\text{WoE}_i = \text{logit } \hat{\pi} - \text{logit } \bar{y}_i$.*

Demostración. Por la regla de Bayes, $\frac{\mathbb{P}(Y=1|X=c_i)}{\mathbb{P}(Y=0|X=c_i)} = \frac{\pi B_i}{(1-\pi)G_i}$; tomando logaritmos, $\text{logit } \mathbb{P}(Y = 1 \mid X = c_i) = \text{logit } \pi - \log(G_i/B_i)$. En la muestra, $\frac{\bar{y}_i}{1-\bar{y}_i} = \frac{n_i^{(1)}}{n_i^{(0)}} = \frac{b_i N^{(1)}}{g_i N^{(0)}} = \frac{b_i}{g_i} \cdot \frac{\hat{\pi}}{1-\hat{\pi}}$, y se toman logaritmos. $\square$

Corolario 7.13. *WoE_i es una función estrictamente decreciente de $\bar{y}_i$, el target encoding sin suavizar. Ambas codificaciones ordenan los niveles de la misma manera (en sentido inverso), el Teorema 7.10 se aplica igual a las dos, y un árbol produce las mismas hojas con cualquiera de ellas.*

Demostración. logit es estrictamente creciente en $(0, 1)$ y $\text{WoE}_i = \text{logit } \hat{\pi} - \text{logit } \bar{y}_i$, luego $-\text{WoE}_i$ es una función estrictamente creciente de $\bar{y}_i$ y la Proposición 1.7 da las mismas hojas con $\bar{y}$ y con $-\text{WoE}$. Falta ver que WoE y $-\text{WoE}$ dan las mismas hojas. Si $v_1 < \dots < v_r$ son los valores distintos de una columna en un nodo, sus cortes producen las particiones $\{\{k : v \leq v_i\}, \{k : v > v_i\}\}$, $i < r$ (Lema 1.6); los de la columna opuesta producen $\{\{k : v \geq v_{i+1}\}, \{k : v < v_{i+1}\}\}$, que como particiones no ordenadas son las mismas. Como el criterio y el desempate de la Definición 1.5 solo dependen de la partición no ordenada, el argumento de la Proposición 1.7 se aplica sin cambios. $\square$

La diferencia está en la escala: el *target encoding* está en la escala de probabilidad y el WoE en la de log-momios, que es la escala lineal de la regresión logística.

Proposición 7.14 (Regresión logística sobre el WoE). *Supóngase $0 < \bar{y}_i < 1$ para todo i . En la regresión logística de y sobre la columna $\text{WoE}_{i(k)}$, los coeficientes $(\hat{\beta}_0, \hat{\beta}_1) = (\text{logit } \hat{\pi}, -1)$ maximizan la verosimilitud, y las probabilidades ajustadas son las proporciones $\bar{y}_i$.*

Demostración. Todo modelo que asigne a las observaciones del nivel c_i una misma probabilidad p_i tiene log-verosimilitud $\sum_i n_i (\bar{y}_i \log p_i + (1 - \bar{y}_i) \log(1 - p_i))$, con $n_i = n_i^{(0)} + n_i^{(1)}$. Cada sumando es una función estrictamente cóncava de $p_i \in (0, 1)$ con derivada nula en $p_i = \bar{y}_i$, así que el máximo sobre todos esos modelos se alcanza en $p_i = \bar{y}_i$. La regresión logística sobre la columna WoE asigna a cada nivel una probabilidad común, luego su verosimilitud no supera ese máximo; con $(\text{logit } \hat{\pi}, -1)$ da $\text{logit } p_i = \text{logit } \hat{\pi} - \text{WoE}_i = \text{logit } \bar{y}_i$ (Proposición 7.12) y lo alcanza. $\square$

Proposición 7.15 (WoE y clasificador bayesiano ingenuo). Sean $X_1, \dots, X_p$ categóricas condicionalmente independientes dado Y , con $\mathbb{P}(X_j = x_j \mid Y = y) > 0$, y $W_j(x_j) = \log \frac{\mathbb{P}(X_j=x_j|Y=0)}{\mathbb{P}(X_j=x_j|Y=1)}$. Entonces

$$\text{logit } \mathbb{P}(Y = 1 \mid X_1 = x_1, \dots, X_p = x_p) = \text{logit } \pi - \sum_{j=1}^p W_j(x_j).$$

Demostración. Por la independencia condicional, $\mathbb{P}(X_1 = x_1, \dots, X_p = x_p \mid Y = y) = \prod_j \mathbb{P}(X_j = x_j \mid Y = y)$. Por la regla de Bayes, el cociente $\mathbb{P}(Y = 1 \mid x) / \mathbb{P}(Y = 0 \mid x)$ es $\frac{\pi}{1-\pi} \prod_j \frac{\mathbb{P}(X_j=x_j|Y=1)}{\mathbb{P}(X_j=x_j|Y=0)}$, cuyo logaritmo es $\text{logit } \pi - \sum_j W_j(x_j)$. $\square$

Una regresión logística sobre las columnas WoE $_j$ con coeficientes libres β_j contiene como caso particular ($\beta_j = -1$ para todo j) al clasificador bayesiano ingenuo; coeficientes alejados de -1 corrigen la dependencia entre variables. Esa aditividad, un término por variable en la escala de log-momios, es la que permite traducir el modelo a una tabla de puntos.

Proposición 7.16 (El IV es una divergencia simétrica). $\text{IV} = \text{KL}(g\|b) + \text{KL}(b\|g)$, donde $\text{KL}(g\|b) = \sum_i g_i \log(g_i/b_i)$ es la divergencia de Kullback–Leibler. En consecuencia $\text{IV} \geq 0$, con igualdad si y solo si $g_i = b_i$ para todo i . Además, cada sumando $(g_i - b_i) \text{WoE}_i$ es ≥ 0 .

Demostración. La primera igualdad es separar $\sum_i (g_i - b_i) \log(g_i/b_i)$ en $\sum_i g_i \log(g_i/b_i) + \sum_i b_i \log(b_i/g_i)$. Por la desigualdad de Jensen para la función cóncava $\log$ y la distribución (g_i) , $-\text{KL}(g\|b) = \sum_i g_i \log(b_i/g_i) \leq \log \sum_i g_i (b_i/g_i) = \log \sum_i b_i = 0$, con igualdad si y solo si b_i/g_i es constante, y la constante es 1 porque $\sum_i g_i = \sum_i b_i = 1$; lo mismo para $\text{KL}(b\|g)$. En cuanto al signo, $g_i - b_i$ y $\log(g_i/b_i)$ son ambos positivos, ambos negativos o ambos nulos. $\square$

La condición $g_i = b_i$ para todo i significa que en la muestra la distribución de X es la misma entre buenos y malos: independencia empírica. De aquí sale un sesgo del IV estimado. Supóngase que en la población X e Y son independientes, de modo que el IV poblacional es 0, que $K \geq 2$ y que todas las combinaciones (nivel, clase) tienen probabilidad positiva. El IV muestral es ≥ 0 , y hay configuraciones de probabilidad positiva en que no vale 0 (por ejemplo, todos los buenos en c_1 y todos los malos en c_2), así que su esperanza es estrictamente positiva. Como cada nivel aporta un sumando no negativo, más niveles dan más ocasiones de sumar ruido positivo. Los umbrales usados en la industria (Siddiqi: < 0.02 inútil, 0.02 – 0.1 débil, 0.1 – 0.3 medio, > 0.3 fuerte) son convenciones empíricas.

Ejemplo 7.17. Mil solicitudes con 100 malos ($\hat{\pi} = 0.1$) y una variable con tres niveles:

Nivel	n_i	malos	$\bar{y}_i$	g_i	b_i	WoE_i	$(g_i - b_i) \text{WoE}_i$
A	500	30	0.060	0.5222	0.30	0.554	0.123
B	400	50	0.125	0.3889	0.50	−0.251	0.028
C	100	20	0.200	0.0889	0.20	−0.811	0.090
IV							0.241

Para A, $g = 470/900$ y $b = 30/100$. Se verifica la Proposición 7.12: $\text{logit } 0.1 - \text{logit } 0.06 = -2.197 - (-2.752) = 0.554$. El *target encoding* con $m = 10$ da (0.0608, 0.1244, 0.1909): el mismo orden, con el nivel pequeño C más contraído hacia 0.1; con $m = 100$, (0.0667, 0.12, 0.15).

Contraste entre *target encoding* y WoE.

	<i>Target encoding</i>	WoE
Respuesta	binaria, multiclase o continua	binaria
Escala	probabilidad o media	log-momios
Suavizado	media posterior; m estimable (Prop. 7.3)	+0.5 en los conteos o fusión de niveles
Modelo natural	árboles (Teor. 7.10) y cualquier otro	regresión logística (Prop. 7.14 y 7.15)
Variables continuas	no se aplica	tras discretizar
Diagnóstico asociado	—	IV por variable
Fuga	sí; requiere codificación cruzada	sí; el mismo tratamiento

Ambas inducen el mismo orden de niveles (Corolario 7.13).

Implementación en AWS. No hay transformación de WoE ni de IV en Data Wrangler, DataBrew ni Canvas, y scikit-learn tampoco la incluye. Una implementación consta de: (i) discretizar las continuas con BUCKETIZATION de DataBrew o KBinsDiscretizer; (ii) un trabajo de **SageMaker Processing** (o de **AWS Glue** en PySpark para volúmenes grandes) que calcule con entrenamiento la tabla de conteos y el WoE, con 0.5 sumado a cada conteo, y la guarde en S3 o en Feature Store:

```
import numpy as np, pandas as pd
def tabla_woe(x, y, eps=0.5):
    t = pd.crosstab(x, y)                                # columnas: 0 (buenos), 1 (malos)
    g = (t[0] + eps) / (t[0] + eps).sum()
    b = (t[1] + eps) / (t[1] + eps).sum()
    woe = np.log(g / b)
    return woe, ((g - b) * woe).sum()                    # (WoE por nivel, IV)
woe, iv = tabla_woe(train["nivel"], train["malo"])
train["nivel_woe"] = train["nivel"].map(woe)
test["nivel_woe"] = test["nivel"].map(woe).fillna(0.0)
```

A un nivel no visto se le asigna 0: por la Proposición 7.12, $WoE = 0$ deja la probabilidad en su valor a priori, es decir, no aporta evidencia. (iii) La regresión logística, con el algoritmo integrado **Linear Learner** (`predictor_type="binary_classifier"`) o con scikit-learn. Con alta cardinalidad se usa la codificación cruzada de la Proposición 7.7.

7.3. Resumen de codificaciones categóricas

Codificación	d	rango[1 Φ]	Usa y	Observaciones
Ordinal	1	2	no	efectos lineales en el orden elegido
<i>One-hot</i>	$K (K - 1)$	K	no	saturada; niveles equidistantes; varianza σ^2/n_i
Binaria	$\lceil \log_2(K + 1) \rceil$	$\leq d + 1$	no	inyectiva; similitudes fijadas por la numeración
<i>Hashing</i>	m	$\leq m + 1$	no	colisiones; sin diccionario; símbolos nuevos sin costo
Similitud	d fijo	—	no	coincidencias según 3-gramas comunes
<i>Target encoding</i>	1 (o C)	2	sí	orden óptimo para árboles; codificación cruzada
WoE	1	2	sí	escala logit; respuesta binaria

8. Series de tiempo

Sea $(y_t)_{t \in \mathbb{Z}}$ un proceso real observado en instantes equiespaciados $t = 1, \dots, T$, y sea $\mathcal{H}_t = (y_s)_{s < t}$ su *historia* antes de t . Para predecir y_t solo pueden usarse funciones de $\mathcal{H}_t$ y funciones

conocidas de t (el calendario). Una característica que dependa de y_t o de valores posteriores es fuga de información: en producción no existe cuando hay que predecir.

8.1. Descomposición de la marca de tiempo

De la marca de tiempo de t se extraen componentes de calendario: mes, día del mes, día de la semana, hora, trimestre. Cada una es una variable *cíclica*: toma valores $u \in \{0, 1, \dots, P-1\}$ y después de $P-1$ vuelve a 0 (mes: $P = 12$; hora: $P = 24$). La codificación ordinal $\phi(u) = u$ coloca a los valores $P-1$ y 0 (diciembre y enero) a la máxima distancia, aunque son consecutivos.

Definición 8.1 (Codificación cíclica). $e(u) = (\sin(2\pi u/P), \cos(2\pi u/P)) \in \mathbb{R}^2$. La *distancia circular* entre u y v es $\delta(u, v) = \min(|u - v|, P - |u - v|)$.

Proposición 8.2. $\|e(u) - e(v)\| = 2 \sin(\pi \delta(u, v)/P)$; en particular, la *distancia euclídea* entre codificaciones es una función estrictamente creciente de la *distancia circular*.

Demostración. Con $a = 2\pi u/P$ y $b = 2\pi v/P$, $\|e(u) - e(v)\|^2 = 2 - 2(\sin a \sin b + \cos a \cos b) = 2 - 2\cos(a - b) = 4\sin^2(\frac{a-b}{2})$, de modo que $\|e(u) - e(v)\| = 2|\sin(\pi(u-v)/P)|$. Sea $\Delta = |u - v| \in [0, P)$. Como $\sin(\pi - x) = \sin x$, $\sin(\pi\Delta/P) = \sin(\pi(P - \Delta)/P)$, así que el valor coincide en Δ y en $P - \Delta$, y por tanto en $\delta(u, v) \in [0, P/2]$; en $[0, P/2]$, $\delta \mapsto \sin(\pi\delta/P)$ es no negativa y estrictamente creciente. $\square$

Con meses, diciembre y enero quedan a distancia $2\sin(\pi/12) \approx 0.52$; enero y junio, a $2\sin(5\pi/12) \approx 1.93$. Un predictor lineal sobre $e(u)$ es $\beta_0 + \beta_1 \sin a + \beta_2 \cos a$ con $a = 2\pi u/P$; con $R = \sqrt{\beta_1^2 + \beta_2^2}$ y φ tal que $\cos \varphi = \beta_2/R$ y $\sin \varphi = \beta_1/R$ (si $R > 0$), es igual a $\beta_0 + R \cos(a - \varphi)$ por la fórmula del coseno de una diferencia: una sola sinusoide de período P . Perfiles estacionales más generales requieren más armónicos, $(\sin(2\pi hu/P), \cos(2\pi hu/P))$ con $h = 2, 3, \dots$, o *one-hot* de u , saturado con $P-1$ columnas (Proposición 6.2).

Para árboles la codificación cíclica es incómoda. Como $\sin(2\pi(P/2 - u)/P) = \sin(\pi - 2\pi u/P) = \sin(2\pi u/P)$, los valores u y $P/2 - u$ (módulo P) tienen el mismo seno, y ningún corte sobre esa coordenada los separa; análogamente, u y $P - u$ tienen el mismo coseno. Con $P = 12$, los cortes sobre el seno no separan enero ($u = 1$) de mayo ($u = 5$), y los cortes sobre el coseno no separan enero de noviembre ($u = 11$): aislar enero exige cortar en las dos coordenadas. Con la codificación ordinal bastan dos cortes sobre una. La documentación de Data Wrangler recomienda el modo cíclico para modelos lineales y redes, y el ordinal para árboles, en coherencia con este análisis.

Implementación en AWS. **Data Wrangler**, *Featurize datetime*: acepta cadenas de fecha o marcas Unix, infiere o recibe el formato (`strftime`) y produce los componentes en modo *Cyclic* u *Ordinal*, como vector o en columnas. Para tratar un componente como categórico se encadena *Encode categorical* $\rightarrow$ *One-hot encode* (por ejemplo, el trimestre en cuatro columnas). En **SageMaker Processing**, con `pandas.Series.dt` y las fórmulas anteriores.

8.2. Características de rezago

Definición 8.3. El *rezago* de orden $h \geq 1$ en el instante t es y_{t-h} ; las características de rezago de orden 1 a p son $(y_{t-1}, \dots, y_{t-p})$, y los rezagos estacionales son $y_{t-P}, y_{t-2P}, \dots$

Su justificación proviene de los modelos autorregresivos.

Proposición 8.4. Supóngase $\mathbb{E}[y_t^2] < \infty$ y $y_t = \phi_0 + \sum_{h=1}^p \phi_h y_{t-h} + \varepsilon_t$, con ε_t de media 0 e independiente de $\mathcal{H}_t$. Entre todas las funciones f de la historia con $\mathbb{E}[f^2] < \infty$, la que minimiza $\mathbb{E}[(y_t - f)^2]$ es $f^* = \phi_0 + \sum_{h=1}^p \phi_h y_{t-h}$.

Demostración. Sea $m_t = \mathbb{E}[y_t \mid \mathcal{H}_t]$. Para cualquier f función de la historia, $\mathbb{E}[(y_t - f)^2] = \mathbb{E}[(y_t - m_t)^2] + \mathbb{E}[(m_t - f)^2] + 2\mathbb{E}[(y_t - m_t)(m_t - f)]$, y el último término es nulo: condicionando en $\mathcal{H}_t$, el factor $m_t - f$ es conocido y $\mathbb{E}[y_t - m_t \mid \mathcal{H}_t] = 0$. Luego $\mathbb{E}[(y_t - f)^2] \geq \mathbb{E}[(y_t - m_t)^2]$, con igualdad si $f = m_t$. Como los y_{t-h} son funciones de la historia y ε_t es independiente de ella, $m_t = \phi_0 + \sum_h \phi_h y_{t-h} + \mathbb{E}[\varepsilon_t] = f^*$. $\square$

Bajo el modelo, los rezagos $1, \dots, p$ contienen toda la información de la historia útil para predecir en error cuadrático, y los de orden mayor no la mejoran. Esto justifica elegir p con la *autocorrelación parcial*: si el proceso es estacionario (medias y covarianzas invariantes en el tiempo) con matrices de covarianza no singulares, la autocorrelación parcial de orden k es el coeficiente de y_{t-k} en el mejor predictor lineal de y_t a partir de $(1, y_{t-1}, \dots, y_{t-k})$. Si $k \geq p+1$, f^* es lineal en esas variables y óptimo entre todas las funciones de la historia, así que también es el mejor predictor lineal a partir de ellas; por la unicidad de este (consecuencia de la no singularidad), el coeficiente de y_{t-k} es 0. Las primeras p observaciones de la serie no tienen historia completa: se descartan o se imputan sus rezagos.

Implementación en AWS. Data Wrangler, Lag features: genera los rezagos hasta una duración dada respecto de una columna de tiempo (por serie, si antes se aplica *Group by*), con opciones para incluir la ventana completa, aplanarla en columnas y descartar las filas sin historia. Antes conviene *Validate timestamps* y *Resample*, para que un rezago corresponda a un intervalo fijo. En pandas, `groupby(...).shift(h)`.

8.3. Características de ventana móvil

Definición 8.5. Para una ventana de tamaño w que termina en $t-1$, la media móvil es $\bar{y}_t^{(w)} = w^{-1} \sum_{h=1}^w y_{t-h}$; análogamente se definen el mínimo, el máximo y la desviación estándar de $(y_{t-w}, \dots, y_{t-1})$.

Que la ventana termine en $t-1$ es la condición de admisibilidad del principio de la sección. El tamaño w resuelve un compromiso entre sesgo y varianza.

Proposición 8.6. Sea $y_t = a + bt + \varepsilon_t$ con ε_t independientes de media 0 y varianza σ^2 . Entonces

$$\mathbb{E}[(y_t - \bar{y}_t^{(w)})^2] = \frac{b^2(w+1)^2}{4} + \frac{\sigma^2}{w} + \sigma^2.$$

Demostración. Como $\sum_{h=1}^w h = w(w+1)/2$, $\bar{y}_t^{(w)} = a + b(t - \frac{w+1}{2}) + \bar{\varepsilon}$ con $\bar{\varepsilon} = w^{-1} \sum_{h=1}^w \varepsilon_{t-h}$. Entonces $y_t - \bar{y}_t^{(w)} = \frac{b(w+1)}{2} + \varepsilon_t - \bar{\varepsilon}$, donde ε_t y $\bar{\varepsilon}$ son independientes (involucran errores de instantes distintos), de media 0, con varianzas σ^2 y σ^2/w . El error cuadrático medio es el cuadrado de la constante más la suma de las varianzas. $\square$

Sin tendencia ($b = 0$) el error decrece con w ; con tendencia, el término de sesgo crece como w^2 . Derivando respecto de w la parte que depende de w , el mínimo cumple $b^2(w+1)/2 = \sigma^2/w^2$, y para w grande $w^* \approx (2\sigma^2/b^2)^{1/3}$: la ventana debe acortarse cuando la tendencia crece respecto del ruido. El mínimo, el máximo y la desviación estándar móviles describen el rango y la volatilidad recientes, que la media no refleja. La extracción automática de muchos estadísticos de ventana multiplica las columnas y conviene seguirla de selección (§11).

Contraste. Los rezagos representan exactamente la dinámica de un modelo autorregresivo y permiten ponderar cada retardo; su número crece con la memoria que se quiera capturar. Las ventanas resumen muchos retardos en pocos números, con menos varianza y pérdida de detalle. Las componentes de fecha no dependen de y , así que no producen fuga y existen para cualquier horizonte; un rezago de orden h solo está disponible para predecir hasta h pasos adelante sin usar predicciones intermedias.

Implementación en AWS. *Data Wrangler*, *Rolling window features*: tamaño de ventana, columna de tiempo y conjunto de estadísticos (mínimo, eficiente, completo o elegido a mano), con opción de aplanar; *Extract features* calcula estadísticos de la serie completa. *Split data* en modo *Ordered* deja la prueba después del entrenamiento. En pandas, `y.shift(1).rolling(w).mean()`: el `shift(1)` hace que la ventana termine en $t - 1$.

9. Imágenes

Una imagen digital de $H \times W$ píxeles y C canales (tres en color: rojo, verde, azul) es un arreglo $I \in \{0, \dots, 255\}^{H \times W \times C}$; como vector, está en $\mathbb{R}^{HWC}$, y con 224×224 píxeles en color tiene 150 528 coordenadas. Usar esas coordenadas como columnas tiene dos problemas: la dimensión (§9.4) y la sensibilidad a desplazamientos, porque mover un objeto un píxel cambia casi todas las coordenadas. Las técnicas de esta sección atacan uno u otro.

9.1. Normalización de píxeles

Dividir por 255 es un escalado MinMax (Definición 3.1) con los límites posibles 0 y 255 en lugar de los observados: no se estima nada, no hay fuga y un píxel extremo no comprime a los demás (a diferencia de la Proposición 3.4). La estandarización por canal, $(I_c/255 - \mu_c)/\sigma_c$, usa medias y desviaciones fijas de un corpus de referencia. Con una red preentrenada deben usarse las del preentrenamiento (en ImageNet, $\mu = (0.485, 0.456, 0.406)$ y $\sigma = (0.229, 0.224, 0.225)$): la red se ajustó sobre entradas con esa escala, y otra escala en inferencia es un caso de *training-serving skew*.

9.2. Extracción clásica: convolución y gradientes

Para enunciar las propiedades sin efectos de borde, se considera una imagen de un canal como una función $I : \mathbb{Z}^2 \rightarrow \mathbb{R}$ nula fuera de un conjunto finito (la imagen real, extendida con ceros).

Definición 9.1. Sea $K : \mathbb{Z}^2 \rightarrow \mathbb{R}$ nula fuera de un conjunto finito (el *núcleo*). La convolución, en la convención usada en redes neuronales, es $(I * K)(p) = \sum_{q \in \mathbb{Z}^2} I(p + q)K(q)$. El núcleo de Sobel horizontal es $K_x(q_1, q_2) = q_1 w(q_2)$ para $q_1, q_2 \in \{-1, 0, 1\}$, con $w(\pm 1) = 1$ y $w(0) = 2$, y 0 en otro caso; en forma de matriz, $\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$.

K_x combina una diferencia centrada en la primera coordenada con un promedio ponderado $(1, 2, 1)$ en la segunda. En un borde vertical, $I(p) = \mathbf{1}\{p_1 \geq 0\}$, se obtiene $(I * K_x)(p) = \sum_{q_2} w(q_2) \sum_{q_1} q_1 \mathbf{1}\{p_1 + q_1 \geq 0\} = 4(\mathbf{1}\{p_1 \geq -1\} - \mathbf{1}\{p_1 \geq 1\}) = 4 \cdot \mathbf{1}\{-1 \leq p_1 \leq 0\}$: la respuesta es 4 en las dos columnas contiguas al borde y 0 en el resto.

Proposición 9.2 (Equivariancia a traslaciones). Para $a \in \mathbb{Z}^2$, sea $(\tau_a I)(p) = I(p - a)$. Entonces $(\tau_a I) * K = \tau_a(I * K)$. Si $\rho : \mathbb{R} \rightarrow \mathbb{R}$ cumple $\rho(0) = 0$, la cantidad $F(I) = \sum_{p \in \mathbb{Z}^2} \rho((I * K)(p))$ es una suma finita y $F(\tau_a I) = F(I)$.

Demostración. $((\tau_a I) * K)(p) = \sum_q I(p + q - a)K(q) = (I * K)(p - a) = (\tau_a(I * K))(p)$. La función $I * K$ es nula fuera de un conjunto finito (el soporte de I más el de K), y $\rho(0) = 0$, así que la suma que define F tiene finitos términos no nulos. Por la primera parte, $F(\tau_a I) = \sum_p \rho((I * K)(p - a))$, que es la misma suma reindexada con $p' = p - a$. $\square$

Un detector de bordes responde igual a un borde en cualquier posición y se desplaza con él; agregando sus respuestas se obtienen características que no cambian al desplazar el contenido. En una imagen recortada a $H \times W$ la igualdad vale mientras el desplazamiento no saque contenido del recuadro. Los histogramas son el caso extremo de agregación:

Ejemplo 9.3 (Histograma y su límite). El histograma $h_I(v) = \#\{p : I(p) = v\}$, $v \in \{0, \dots, 255\}$, de una imagen de $H \times W$ píxeles no cambia si los píxeles se permutan, porque solo cuenta valores; en particular, no cambia con traslaciones cíclicas ni con rotaciones de 90° . Por la misma razón pierde toda la disposición espacial: un tablero de 8×8 casillas blancas y negras y una imagen con la mitad izquierda negra y la derecha blanca, del mismo tamaño, tienen el mismo histograma.

9.3. Vectores de características de una red preentrenada

Una red convolucional (CNN) compone capas de convolución con núcleos aprendidos (equivariantes por la Proposición 9.2), funciones no lineales y agregaciones espaciales, y termina en una capa lineal seguida de *softmax*. Si $\psi : \mathbb{R}^{H \times W \times C} \rightarrow \mathbb{R}^D$ denota la salida de la penúltima capa, la red calcula probabilidades de clase $\text{softmax}(A\psi(I) + c)$ con $A \in \mathbb{R}^{C' \times D}$ y $c \in \mathbb{R}^{C'}$, que es un modelo logístico multinomial con características $\psi(I)$; al entrenar la red se ajustan conjuntamente ψ y (A, c) . La *transferencia* consiste en tomar ψ de una red entrenada en un corpus grande, calcular $\psi(I)$ (D del orden de 10^3) para las imágenes de un problema nuevo y entrenar sobre esas columnas un modelo sencillo. Que la ψ aprendida para unas clases sirva para otras no es un teorema, sino una observación empírica: Yosinski et al. (2014) midieron que las capas iniciales aprenden detectores genéricos (bordes, texturas) y que la transferibilidad disminuye en las capas finales y cuanto más difieren las tareas.

Contraste. Las características clásicas (gradientes, histogramas) son interpretables, baratas y sin parámetros aprendidos, pero no representan objetos. Los vectores de una CNN sí, a costa de interpretabilidad, cómputo y dependencia del preprocesamiento del preentrenamiento. Las etiquetas de un servicio de visión son pocas columnas legibles, limitadas al vocabulario del servicio.

Implementación en AWS. SageMaker JumpStart ofrece modelos de *image embedding* (por ejemplo, la familia *Image Embedding – TensorFlow*) que devuelven $\psi(I)$ para cada imagen, desplegables como *endpoint* o en *batch transform*. Amazon Bedrock ofrece modelos de *embeddings* multimodales. Amazon Rekognition (DetectLabels) devuelve etiquetas con su confianza y, para objetos, *bounding boxes* con Left, Top, Width y Height expresados como fracciones del tamaño de la imagen; con ellas se construyen columnas como la indicadora de una etiqueta con confianza mayor que un umbral, el número de instancias o la fracción de área $\text{Width} \cdot \text{Height}$. (Las funciones de *Streaming Events* y de moderación de contenido por lotes de Rekognition dejaron de aceptar clientes nuevos el 30 de abril de 2026; la detección de etiquetas no está afectada.) Data Wrangler incluye transformaciones de imagen (cambio de tamaño, mejora, escala de grises, rotación, eliminación de imágenes corruptas o duplicadas) y, sobre las características ya tabulares, normalización y PCA.

9.4. Análisis de componentes principales

El capítulo propone PCA para reducir la dimensión de las características de imagen y la de *one-hot*. Sea $X \in \mathbb{R}^{n \times p}$ con columnas de media cero y $S = X^\top X/n$ su matriz de covarianzas muestrales. Por el teorema espectral, $S = \sum_{i=1}^p \lambda_i u_i u_i^\top$ con $\lambda_1 \geq \dots \geq \lambda_p \geq 0$ y $\{u_1, \dots, u_p\}$ una base ortonormal de autovectores; los autovalores son no negativos porque $u^\top S u = \|Xu\|^2/n \geq 0$.

Teorema 9.4 (PCA como máxima varianza). Para $1 \leq d \leq p$, el máximo de $\text{tr}(V^\top S V)$ sobre las matrices $V \in \mathbb{R}^{p \times d}$ con $V^\top V = I_d$ es $\lambda_1 + \dots + \lambda_d$ y se alcanza en $V = [u_1 \dots u_d]$. La cantidad $\text{tr}(V^\top S V)$ es la suma de las varianzas muestrales de las columnas de XV , las proyecciones de los datos sobre las columnas de V .

Demostración. $\text{tr}(V^\top SV) = \sum_i \lambda_i \text{tr}(V^\top u_i u_i^\top V) = \sum_i \lambda_i w_i$ con $w_i = \|V^\top u_i\|^2$. Por un lado, $VV^\top$ es la proyección ortogonal sobre el espacio columna de V (porque $V^\top V = I$), así que $w_i = u_i^\top VV^\top u_i = \|VV^\top u_i\|^2 \leq \|u_i\|^2 = 1$. Por otro, $\sum_i w_i = \text{tr}(V^\top (\sum_i u_i u_i^\top) V) = \text{tr}(V^\top V) = d$. Como $w_i - 1 \leq 0$ y $\lambda_i \geq \lambda_d$ para $i \leq d$, y $w_i \geq 0$ y $\lambda_i \leq \lambda_d$ para $i > d$,

$$\sum_i \lambda_i w_i - \sum_{i \leq d} \lambda_i = \sum_{i \leq d} \lambda_i (w_i - 1) + \sum_{i > d} \lambda_i w_i \leq \lambda_d \left(\sum_{i \leq d} (w_i - 1) + \sum_{i > d} w_i \right) = \lambda_d \left(\sum_i w_i - d \right) = 0.$$

Con $V = [u_1 \cdots u_d]$ se tiene $w_i = 1$ para $i \leq d$ y $w_i = 0$ para $i > d$, y se alcanza el valor. Finalmente, las columnas de XV tienen media cero (porque las de X la tienen), su matriz de covarianzas es $V^\top X^\top XV/n = V^\top SV$, y la traza es la suma de sus varianzas. $\square$

Proposición 9.5 (Error de reconstrucción). *Con $V = [u_1 \cdots u_d]$, el error cuadrático medio de reconstruir cada fila x_l por su proyección $VV^\top x_l$ es $n^{-1} \sum_l \|x_l - VV^\top x_l\|^2 = \sum_{i > d} \lambda_i$.*

Demostración. Como $VV^\top$ es una proyección ortogonal, $x - VV^\top x$ es ortogonal a $VV^\top x$ y, por el teorema de Pitágoras, $\|x - VV^\top x\|^2 = \|x\|^2 - \|VV^\top x\|^2 = \|x\|^2 - \|V^\top x\|^2$ (la última igualdad porque $\|Vv\| = \|v\|$ cuando $V^\top V = I$). Promediando sobre las filas, $n^{-1} \sum_l \|x_l\|^2 = \text{tr}(S) = \sum_i \lambda_i$ y $n^{-1} \sum_l \|V^\top x_l\|^2 = \text{tr}(V^\top SV) = \sum_{i \leq d} \lambda_i$. $\square$

La fracción de varianza explicada por d componentes es $\sum_{i \leq d} \lambda_i / \sum_{i \leq p} \lambda_i$, y lo que falta es, por la proposición, el error relativo de reconstrucción. En la práctica se usa la descomposición en valores singulares $X = U\Sigma W^\top$ (U y W con columnas ortonormales, Σ diagonal con valores singulares $\sigma_i \geq 0$): entonces $S = W(\Sigma^2/n)W^\top$, de modo que las columnas de W son autovectores de S con autovalores σ_i^2/n , y no hace falta formar S .

Ejemplo 9.6. Si $S = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, los autovalores son 3 y 1, con autovectores $(1, 1)/\sqrt{2}$ y $(1, -1)/\sqrt{2}$; la primera componente, la suma normalizada de las dos variables, explica 3/4 de la varianza total.

Observación 9.7 (PCA depende de las unidades). Si una columna se multiplica por c , sus entradas en S se multiplican por c (covarianzas) o c^2 (varianza); medir una longitud en centímetros en lugar de metros multiplica su varianza por 10^4 y la primera componente tiende a alinearse con ella. Estandarizar antes equivale a hacer PCA sobre la matriz de correlaciones. Tras *one-hot*, la indicadora de un nivel de frecuencia relativa f_i tiene varianza $f_i(1 - f_i)$ (varianza de una Bernoulli), pequeña para niveles raros; PCA, que maximiza varianza y no usa y , tiende a descartarlos aunque sean predictivos.

Implementación en AWS. Data Wrangler, *Reduce dimensionality* (PCA): número de componentes fijo o umbral de fracción de varianza explicada (95 % por defecto), opciones de centrar y de escalar a desviación 1 (la observación anterior indica cuándo usarlas), salida como vector o columnas y opción de conservar las columnas originales. **Algoritmo integrado PCA** de SageMaker: modo *regular* (datos dispersos, tamaño moderado) o *randomized* (muchas filas y columnas, con un algoritmo aproximado); calcula la matriz de covarianzas, o una aproximación distribuida, y su descomposición en valores singulares; acepta `recordIO-protobuf` y CSV.

10. Texto

Un *corpus* es una colección de documentos $d_1, \dots, d_N$, cada uno una cadena de caracteres. El objetivo es representar cada documento (o cada palabra) por un vector. La secuencia habitual es: tokenizar, normalizar (palabras vacías, *stemming* o lematización) y representar (bolsa de palabras, n -gramas, TF-IDF o *embeddings*).

10.1. Tokenización

Definición 10.1. Un *tokenizador* es una función que asigna a cada cadena de caracteres una sucesión finita $(t_1, \dots, t_L)$ de elementos de un conjunto $\mathcal{V}$, el *vocabulario*; los t_ℓ son los *tokens* y L la longitud del documento en tokens. En la tokenización por palabras, los tokens son las subcadenas delimitadas por espacios y signos de puntuación, y $\mathcal{V}$ es el conjunto de palabras vistas en entrenamiento.

Por ejemplo, «Machine learning is fascinating» da (Machine, learning, is, fascinating). Con tokenización por palabras, una palabra ausente de $\mathcal{V}$ no tiene representación y se sustituye por un símbolo genérico, con lo que su información se pierde. Los modelos fundacionales usan tokenización por subpalabras, que elimina ese problema.

Definición 10.2 (*Byte-pair encoding*, Sennrich et al., 2016). Sea $\mathcal{V}_0$ el conjunto de caracteres (o de bytes). Se segmenta cada palabra del corpus de entrenamiento en caracteres y se repite M veces: contar, en la segmentación actual, la frecuencia de cada par de símbolos adyacentes; agregar al vocabulario el símbolo formado por la concatenación del par más frecuente (con una regla fija de desempate); reemplazar en la segmentación todas sus apariciones. El vocabulario final tiene $|\mathcal{V}_0| + M$ elementos, y la lista ordenada de fusiones define el tokenizador: una cadena nueva se segmenta en caracteres y se le aplican las fusiones en el mismo orden.

Proposición 10.3. Si $\mathcal{V}_0$ contiene todos los bytes, toda cadena se tokeniza con símbolos de $\mathcal{V}$.

Demostración. La segmentación inicial usa solo elementos de $\mathcal{V}_0 \subset \mathcal{V}$, y cada fusión reemplaza dos símbolos consecutivos por su concatenación, que es un elemento de $\mathcal{V}$ por construcción; por inducción sobre el número de fusiones aplicadas, todos los símbolos de la segmentación están en $\mathcal{V}$. $\square$

Las palabras frecuentes terminan como un solo token y las raras se dividen en fragmentos. El número de tokens por palabra depende del idioma y del tokenizador, y determina el costo en servicios de modelos fundacionales de texto facturados por uso: $p_{\text{in}}T_{\text{in}} + p_{\text{out}}T_{\text{out}}$, con $T_{\text{in}}, T_{\text{out}}$ los números de tokens de entrada y salida y $p_{\text{in}}, p_{\text{out}}$ sus precios unitarios.

Implementación en AWS. Glue DataBrew, acción TOKENIZATION, que en un paso tokeniza con un delimitador, expande contracciones, aplica *stemming* y elimina palabras vacías:

```
{"Action": {"Operation": "TOKENIZATION",
  "Parameters": {"sourceColumn": "resena", "targetColumn": "resena_tok",
    "delimiter": " ", "expandContractions": "ENABLED",
    "stemmingMode": "PORTER", "stopWordRemovalMode": "DEFAULT",
    "customStopWords": "[]"}}}
```

Data Wrangler, *Featurize text* $\rightarrow$ *Vectorize*, con tokenizador *Standard* (separación por espacios) o *Custom* (expresión regular). En **Amazon Bedrock** la tokenización por subpalabras es interna a cada modelo y es la unidad de facturación de los modelos de texto bajo demanda.

10.2. Palabras vacías

La *ley de Zipf* es una regularidad empírica: en muchos corpus, la frecuencia de la palabra que ocupa el lugar r en orden de frecuencia es aproximadamente proporcional a r^{-s} , con s cercano a 1. Bajo este modelo con $s = 1$ y V palabras distintas, la fracción de los tokens que corresponde a las k más frecuentes es H_k/H_V , con $H_k = \sum_{r=1}^k 1/r$: la frecuencia de la palabra r es r^{-1}/H_V del total. Con $V = 50\,000$ y $k = 100$, $H_{100}/H_{50\,000} \approx 5.19/11.40 \approx 0.46$. Las palabras más frecuentes son en su mayoría funcionales (artículos, preposiciones, conjunciones), y se llaman *palabras vacías*. Eliminarlas reduce casi a la mitad el volumen de tokens y el peso de términos que aparecen en todos los documentos, pero no es neutral: las listas estándar incluyen

negaciones, y eliminar «no» convierte «no bueno» en «bueno». La ponderación TF-IDF (§10.4) hace una atenuación basada en los datos.

Implementación en AWS. **DataBrew:** `stopWordRemovalMode` (lista por defecto) y `customStopWords` dentro de `TOKENIZATION`. **Data Wrangler:** el *Count vectorizer* admite una frecuencia documental máxima, que elimina los términos presentes en demasiados documentos: una lista de palabras vacías construida con los datos.

10.3. *Stemming* y lematización

Definición 10.4. Una *normalización* es una función $\psi : \mathcal{V} \rightarrow \mathcal{V}'$ que asigna a cada palabra una forma representativa; las palabras con la misma imagen se consideran la misma. El *stemming* define ψ con reglas de supresión de sufijos (algoritmos de Porter, 1980, y de Lancaster); la lematización, con un diccionario morfológico que usa la categoría gramatical de la palabra.

Proposición 10.5 (Normalizar es restringir coeficientes). Sea $x \in \mathbb{R}^{\mathcal{V}}$ el vector de conteos de un documento (x_v es el número de apariciones de la palabra v) y $x' = Mx \in \mathbb{R}^{\mathcal{V}'}$ el vector tras normalizar, con $M_{v'v} = \mathbf{1}\{\psi(v) = v'\}$. Los predictores lineales $\gamma^\top x'$ son exactamente los predictores lineales $\beta^\top x$ cuyos coeficientes son iguales dentro de cada clase $\psi^{-1}(v')$.

Demostración. $\gamma^\top x' = \gamma^\top Mx = (M^\top \gamma)^\top x$ y $(M^\top \gamma)_v = \sum_{v'} M_{v'v} \gamma_{v'} = \gamma_{\psi(v)}$, constante en cada clase. Recíprocamente, si $\beta_u = \beta_v$ siempre que $\psi(u) = \psi(v)$, se toma $\gamma_{v'}$ igual al valor común de β en $\psi^{-1}(v')$ (cualquiera si la clase es vacía) y se tiene $\beta = M^\top \gamma$. $\square$

Normalizar reduce el número de parámetros y agrega los conteos de formas de la misma palabra (menos varianza), a cambio de sesgo cuando se fusionan palabras con efectos distintos. Ejemplos reales (implementaciones de NLTK): Porter lleva «studies» y «studying» a «studi», pero también «university» y «universe» a «univers» y «organization» y «organ» a «organ»; Lancaster, más agresivo, lleva «generous» y «generation» a «gen» y «better» a «bet». La lematización lleva «studies» a «study» y, sabiendo que es un adjetivo, «better» a «good», lo que ninguna regla de supresión de sufijos logra. Porter y Lancaster son algoritmos para inglés; para español se usa el *stemmer* Snowball de español o un lematizador.

Implementación en AWS. **DataBrew:** `stemmingMode` igual a `PORTER` o `LANCASTER` en `TOKENIZATION`. No hay lematización integrada en DataBrew ni en Data Wrangler; se implementa en **SageMaker Processing** con spaCy o NLTK instalados en el contenedor. **Amazon Comprehend** (`DetectSyntax`) devuelve los tokens con su categoría gramatical, que es el dato que necesita un lematizador.

10.4. Bolsa de palabras, n -gramas y TF-IDF

Definición 10.6. Fijado un vocabulario $\mathcal{V}$ de *términos*, la *matriz documento-término* $X \in \mathbb{R}^{N \times |\mathcal{V}|}$ tiene entradas $X_{dt} = \text{tf}_{t,d}$, el número de apariciones del término t en el documento d ; la fila d es la *bolsa de palabras* del documento. Un n -grama es una sucesión de n tokens consecutivos; en la representación por n -gramas, los términos son los n -gramas observados en entrenamiento. Sea df_t el número de documentos de entrenamiento que contienen t . Las ponderaciones TF-IDF de Spark ML y de scikit-learn (con `smooth_idf=True`) son $\text{tf}_{t,d} \cdot \text{idf}_t$ con

$$\text{idf}_t^{\text{Spark}} = \log \frac{N+1}{\text{df}_t+1}, \quad \text{idf}_t^{\text{sklearn}} = \log \frac{N+1}{\text{df}_t+1} + 1,$$

habitualmente seguidas de normalización euclídea de cada fila.

La bolsa de palabras solo cuenta, así que no cambia si se permutan los tokens de un documento: «el perro muerde al hombre» y «el hombre muerde al perro» tienen la misma fila. Con bigramas difieren («perro muerde» frente a «hombre muerde»), y «no bueno» queda como un término. El costo es combinatorio: hay $|\mathcal{V}|^n$ n -gramas posibles y un documento de L tokens contiene a lo sumo $L - n + 1$, así que la matriz es más dispersa al crecer n .

La ponderación IDF (Spärck Jones, 1972) responde a que un término presente en muchos documentos discrimina poco entre ellos: $\log(N/\text{df}_t)$ es la cantidad de información, $-\log p$, del suceso «el término aparece en un documento elegido al azar», cuya frecuencia es $p = \text{df}_t/N$. Los $+1$ evitan divisiones por cero; el $+1$ final de scikit-learn impide que un término presente en todos los documentos reciba peso nulo.

Ejemplo 10.7. Documentos «el gato come», «el perro come carne» y «el gato duerme» ($N = 3$). Con la fórmula de Spark: $\text{idf}(\text{el}) = \log(4/4) = 0$, $\text{idf}(\text{gato}) = \text{idf}(\text{come}) = \log(4/3) \approx 0.288$ y $\text{idf}(\text{perro}) = \text{idf}(\text{carne}) = \text{idf}(\text{duerme}) = \log 2 \approx 0.693$. Con la de scikit-learn, los mismos valores más 1: «el» conserva peso 1.

Proposición 10.8. Si las filas TF-IDF se normalizan a norma euclídea 1, el producto interno de dos filas normalizadas es el coseno del ángulo entre las filas sin normalizar. Si todos los conteos de un documento se multiplican por $c > 0$, su fila normalizada no cambia.

Demostración. $\langle u/\|u\|, v/\|v\| \rangle = \langle u, v \rangle / (\|u\|\|v\|)$, que es la definición del coseno. Si $\text{tf}_{t,d} \mapsto c \text{tf}_{t,d}$, la fila TF-IDF se multiplica por c (los idf son los de entrenamiento y no cambian), y su normalización es la misma. $\square$

Por ejemplo, un documento y su concatenación consigo mismo quedan representados igual: la normalización elimina el efecto de la longitud.

Implementación en AWS. Data Wrangler, Featurize text $\rightarrow$ Vectorize: tokenizador; vectorizador *Count vectorizer* (frecuencias mínima y máxima de término y de documento, tamaño máximo de vocabulario, salida binaria) o *Hashing* (§6.5, caso (b)); opción *Apply IDF* con frecuencia documental mínima (5 por defecto); salida como vector disperso o aplanada. *Character statistics* añade el número de palabras y de caracteres y las proporciones de mayúsculas, minúsculas, dígitos y otros símbolos. Es un paso con parámetros ajustados (vocabulario, IDF), con *refit*. **SageMaker Processing:** `TfidfVectorizer(ngram_range=(1,2), min_df=..., max_df=...)`; si después se escala, `MaxAbsScaler` conserva la dispersión (Proposición 3.6).

10.5. Word embeddings

En la bolsa de palabras cada término es una coordenada, lo que equivale a representar la palabra v por e_v ; para $u \neq v$, $\cos(e_u, e_v) = 0$. «Gato» y «felino» son tan distintos como «gato» e «hipoteca», y lo aprendido sobre una palabra no se transfiere a otra. La hipótesis distribucional (Harris, 1954), según la cual palabras que aparecen en contextos parecidos tienen significados parecidos, sugiere representar cada palabra por un vector denso de dimensión D (del orden de 100 a 300) aprendido de las coocurrencias.

Pares palabra-contexto. Se ve el corpus tokenizado como una sucesión $w_1, \dots, w_{N_T}$ de tokens. Con un radio $L \geq 1$, el multiconjunto de pares es $\mathcal{P} = \{(w_a, w_b) : 1 \leq a, b \leq N_T, 0 < |a - b| \leq L\}$: cada token w_a (la *palabra*) con cada token a distancia a lo sumo L (su *contexto*). Sean $\#(w, c)$ el número de veces que el par (w, c) aparece en $\mathcal{P}$, $\#(w) = \sum_c \#(w, c)$, $\#(c) = \sum_w \#(w, c)$ y $|\mathcal{P}|$ el total de pares; las frecuencias relativas $\hat{P}(w, c) = \#(w, c)/|\mathcal{P}|$, $\hat{P}(w) = \#(w)/|\mathcal{P}|$ y $\hat{P}(c) = \#(c)/|\mathcal{P}|$ definen la *información mutua puntual* $\text{PMI}(w, c) = \log \frac{\hat{P}(w, c)}{\hat{P}(w)\hat{P}(c)}$.

Definición 10.9 (*Skip-gram* con muestreo negativo, Mikolov et al., 2013). Se asigna a cada palabra un vector $v_w \in \mathbb{R}^D$ y a cada contexto un vector $u_c \in \mathbb{R}^D$. Con $\sigma(x) = 1/(1 + e^{-x})$, un entero $k \geq 1$ y una distribución P_n sobre contextos (la distribución de ruido), se maximiza

$$\ell = \sum_{w,c} \#(w,c) \log \sigma(u_c^\top v_w) + k \sum_w \#(w) \sum_c P_n(c) \log \sigma(-u_c^\top v_w).$$

El primer término premia que los pares observados tengan producto interno grande; el segundo es la esperanza, para cada aparición de w , de k contextos falsos sorteados con P_n , y premia que tengan producto interno pequeño. El problema que resolvió esta formulación es de cómputo: el modelo original, $p(c | w) = \exp(u_c^\top v_w) / \sum_{c'} \exp(u_{c'}^\top v_w)$, suma sobre todo el vocabulario en cada evaluación, y ℓ solo evalúa $k + 1$ términos por par. Lo que se aprende queda caracterizado así:

Teorema 10.10 (Levy y Goldberg, 2014). *Supóngase que los números $x_{wc} = u_c^\top v_w$ pueden elegirse libremente (lo que requiere D suficientemente grande) y que $\#(w,c) > 0$ y $P_n(c) > 0$ para todos los pares (w,c) que intervienen en ℓ . Entonces el máximo de ℓ se alcanza en*

$$u_c^\top v_w = \log \frac{\#(w,c)}{\#(w) P_n(c)} - \log k.$$

Si $P_n(c) = \hat{P}(c)$, el valor es $\text{PMI}(w,c) - \log k$.

Demostración. Con los x_{wc} libres, $\ell = \sum_{w,c} \ell_{wc}(x_{wc})$ con $\ell_{wc}(x) = \#(w,c) \log \sigma(x) + k \#(w) P_n(c) \log \sigma(-x)$, y se maximiza cada sumando por separado. Como $\frac{d}{dx} \log \sigma(x) = \sigma(-x)$ y $\frac{d}{dx} \log \sigma(-x) = -\sigma(x)$, se tiene $\ell'_{wc}(x) = \#(w,c) \sigma(-x) - k \#(w) P_n(c) \sigma(x)$ y $\ell''_{wc}(x) = -(\#(w,c) + k \#(w) P_n(c)) \sigma(x) \sigma(-x) < 0$; ℓ_{wc} es estrictamente cóncava y su máximo es su único punto crítico. Usando $\sigma(-x)/\sigma(x) = e^{-x}$, la condición $\ell'_{wc}(x) = 0$ es $\#(w,c) e^{-x} = k \#(w) P_n(c)$, es decir, $x = \log \frac{\#(w,c)}{\#(w) P_n(c)} - \log k$. Con $P_n(c) = \#(c)/|\mathcal{P}|$, $\frac{\#(w,c)}{\#(w) \#(c)/|\mathcal{P}|} = \frac{\hat{P}(w,c)}{\hat{P}(w) \hat{P}(c)}$. $\square$

Con D finito, los productos internos no pueden elegirse libremente y el método ajusta una aproximación de rango D a la matriz de PMI desplazada; las palabras con filas de PMI parecidas reciben vectores parecidos, que es el sentido en que el método generaliza. En la práctica P_n es la distribución unigrama elevada a $3/4$ y renormalizada, y el teorema vale con esa P_n .

Definición 10.11 (GloVe, Pennington et al., 2014). Con X_{ij} el número de coocurrencias de las palabras i y j en ventanas del corpus, se minimiza $J = \sum_{i,j: X_{ij} > 0} f(X_{ij}) (v_i^\top \tilde{v}_j + b_i + \tilde{b}_j - \log X_{ij})^2$ sobre vectores $v_i, \tilde{v}_j \in \mathbb{R}^D$ y sesgos $b_i, \tilde{b}_j \in \mathbb{R}$, con $f(x) = (x/x_{\text{máx}})^{3/4}$ si $x < x_{\text{máx}}$ y $f(x) = 1$ si no ($x_{\text{máx}} = 100$).

GloVe hace explícita una factorización del mismo tipo. Si existen vectores con $v_i^\top \tilde{v}_j = \text{PMI}(i,j)$ para todos los pares con $X_{ij} > 0$ (con PMI calculada a partir de X), entonces con $b_i = \log X_i$ y $\tilde{b}_j = \log X_j - \log X_{..}$, donde $X_i = \sum_j X_{ij}$ y $X_{..} = \sum_{i,j} X_{ij}$, cada término vale $\text{PMI}(i,j) + \log X_i + \log X_j - \log X_{..} - \log X_{ij} = 0$ por la definición de PMI, y $J = 0$ es el mínimo: los sesgos absorben las frecuencias marginales y los productos internos representan la PMI. La función f reduce el peso de los pares rarísimos, cuyos conteos son ruidosos, y acota el de los frecuentísimos.

Proposición 10.12 (Promedio de *embeddings*). Sea $E \in \mathbb{R}^{|\mathcal{V}| \times D}$ la matriz cuyas filas son los vectores v_w y x la bolsa de palabras de un documento con L tokens. El promedio de los vectores de sus tokens es $L^{-1} E^\top x$.

Demostración. $\sum_{\ell=1}^L v_{t_\ell} = \sum_w x_w v_w = E^\top x$, agrupando los tokens iguales, pues x_w cuenta las apariciones de w . $\square$

Representar un documento por el promedio de sus *embeddings* es, por tanto, proyectar su bolsa de palabras (dividida por L) con una matriz fija aprendida en otro corpus, y hereda la invariancia al orden de la bolsa de palabras. El análisis semántico latente (LSA), en cambio, proyecta sobre las primeras componentes principales de la matriz TF-IDF del propio corpus (Teorema 9.4). Los *embeddings* contextuales de los modelos basados en transformadores asignan a cada aparición de una palabra un vector que depende de toda la oración, y pueden distinguir sentidos de una misma palabra («banco» para sentarse y «banco» que presta dinero), lo que un vector por palabra no puede hacer por definición.

Contraste. Bolsa de palabras y TF-IDF: interpretables, sin preentrenamiento, dispersas y de alta dimensión, sin similitud entre palabras ni orden. n -gramas: orden local, a costa de dispersión. Word2Vec y GloVe: densos, con similitud entre palabras y transferibles; un vector por palabra, y sin representación para palabras nuevas salvo que se modelen subpalabras. *Embeddings* contextuales: resuelven la polisemia; son costosos de calcular y opacos.

Implementación en AWS. **BlazingText**, algoritmo integrado de SageMaker, implementa Word2Vec en modos `cbow`, `skipgram` (una CPU o una GPU) y `batch_skipgram` (varias CPU), con vectores de subpalabras (*character n -grams*) que dan vectores a palabras fuera del vocabulario. Espera un archivo de texto con una oración por línea y tokens separados por espacios, y produce `vectors.txt` (compatible con Gensim y spaCy) y `vectors.bin`. **Amazon Bedrock**, *Titan Text Embeddings V2* (`amazon.titan-embed-text-v2:0`): *embeddings* contextuales de documento, con dimensión configurable (1024, 512 o 256) y normalización opcional, para entradas de hasta unos 8000 tokens. **Amazon Comprehend**: características de alto nivel por API (sentimiento con puntuaciones por clase, entidades, frases clave, idioma, sintaxis) que se agregan como columnas; su modelado de tópicos dejó de aceptar clientes nuevos el 30 de abril de 2026, y para tópicos quedan los algoritmos integrados **LDA** y **NTM** de SageMaker. **Amazon Textract** convierte documentos escaneados en texto antes de todo lo anterior.

11. Selección de características por filtros

Un *método de filtro* ordena las características según una medida de asociación con y , calculada en entrenamiento, y conserva las primeras. Las medidas difieren en qué dependencia detectan.

Contraejemplo 11.1 (Correlación nula no es independencia). Si $X \sim N(0, 1)$ e $Y = X^2$, entonces $\text{Cov}(X, Y) = \mathbb{E}[X^3] - \mathbb{E}[X]\mathbb{E}[X^2] = 0$ porque X es simétrica, y $\text{Corr}(X, Y) = 0$, aunque Y es función de X . Un filtro por correlación de Pearson descartaría X .

Definición 11.2. Para X, Y discretas con distribución conjunta P_{XY} y marginales P_X, P_Y , la *información mutua* es $I(X; Y) = \sum_{x, y: P_{XY}(x, y) > 0} P_{XY}(x, y) \log \frac{P_{XY}(x, y)}{P_X(x)P_Y(y)}$, es decir, $\text{KL}(P_{XY} \| P_X P_Y)$.

Proposición 11.3. $I(X; Y) \geq 0$, con igualdad si y solo si X e Y son independientes.

Demostración. Por la desigualdad de Jensen para $\log$ (cóncava) y la distribución P_{XY} , $-I(X; Y) = \sum P_{XY} \log \frac{P_X P_Y}{P_{XY}} \leq \log \sum P_{XY} P_X P_Y \leq \log 1 = 0$, con igualdad en la primera desigualdad si y solo si $P_X P_Y / P_{XY}$ es constante donde $P_{XY} > 0$, y en la segunda si y solo si $P_X P_Y$ se anula donde P_{XY} se anula; ambas juntas equivalen a $P_{XY} = P_X P_Y$. $\square$

La información mutua detecta cualquier dependencia, incluida la del contraejemplo tras discretizar (para variables continuas, scikit-learn la estima con vecinos más cercanos). Para dos categóricas está ligada a la prueba de independencia:

Proposición 11.4. En una tabla de contingencia con conteos O_{ij} , total N y conteos esperados bajo independencia $E_{ij} = O_{i\cdot}O_{\cdot j}/N$ (con $O_{i\cdot}, O_{\cdot j}$ los totales de fila y columna), el estadístico de razón de verosimilitudes $G = 2 \sum_{ij: O_{ij} > 0} O_{ij} \log(O_{ij}/E_{ij})$ vale $2N\hat{I}$, donde $\hat{I}$ es la información mutua de la distribución empírica $\hat{p}_{ij} = O_{ij}/N$.

Demostración. $O_{ij}/E_{ij} = \hat{p}_{ij}/(\hat{p}_{i\cdot}\hat{p}_{\cdot j})$ con $\hat{p}_{i\cdot} = O_{i\cdot}/N$ y $\hat{p}_{\cdot j} = O_{\cdot j}/N$; entonces

$$\sum_{ij} O_{ij} \log \frac{O_{ij}}{E_{ij}} = N \sum_{ij} \hat{p}_{ij} \log \frac{\hat{p}_{ij}}{\hat{p}_{i\cdot}\hat{p}_{\cdot j}} = N\hat{I}.$$

□

El estadístico de Pearson $\chi^2 = \sum_{ij} (O_{ij} - E_{ij})^2 / E_{ij}$ aproxima a G cuando los O_{ij} son cercanos a los E_{ij} : con $u = (O_{ij} - E_{ij})/E_{ij}$, el desarrollo $(1 + u) \log(1 + u) = u + u^2/2 + O(u^3)$ da $O_{ij} \log(O_{ij}/E_{ij}) = (O_{ij} - E_{ij}) + \frac{(O_{ij} - E_{ij})^2}{2E_{ij}} + O(|O_{ij} - E_{ij}|^3/E_{ij}^2)$, y al sumar, como $\sum_{ij} (O_{ij} - E_{ij}) = 0$, queda $G = \chi^2 + \sum_{ij} O(|O_{ij} - E_{ij}|^3/E_{ij}^2)$.

Definición 11.5 (Factor de inflación de la varianza). En una matriz de diseño con intercepto, sea R_j^2 el coeficiente de determinación de la regresión de la columna j sobre las demás columnas. $\text{VIF}_j = 1/(1 - R_j^2)$.

Proposición 11.6. En el modelo $y = Z\beta + \varepsilon$ con Z de rango completo por columnas, que incluye la columna $\mathbf{1}$, y $\text{Var } \varepsilon = \sigma^2 I$, el estimador de mínimos cuadrados cumple $\text{Var}(\hat{\beta}_j) = \sigma^2 \text{VIF}_j / \text{SCT}_j$, con $\text{SCT}_j = \sum_k (z_{kj} - \bar{z}_j)^2$, para cada columna j distinta del intercepto.

Demostración. Por el teorema de Frisch–Waugh–Lovell (que requiere solo que Z tenga rango completo, lo que se supone), $\hat{\beta}_j = r_j^\top y / r_j^\top r_j$, donde r_j es el vector de residuos de la regresión de la columna j sobre las demás. Como r_j no depende de y , $\text{Var}(\hat{\beta}_j) = r_j^\top (\sigma^2 I) r_j / (r_j^\top r_j)^2 = \sigma^2 / (r_j^\top r_j)$. Como las demás columnas incluyen $\mathbf{1}$, la definición del coeficiente de determinación da $r_j^\top r_j = (1 - R_j^2) \text{SCT}_j$. □

El VIF mide redundancia, no relevancia: una característica con VIF alto puede ser muy predictiva, pero su coeficiente es inestable. Es la formalización del ejemplo del capítulo de ingresos y ventas que «se mueven en paralelo».

Implementación en AWS. **Data Wrangler** ofrece análisis (no transformaciones) para seleccionar: *Quick Model* (bosque aleatorio con importancias de Gini y partición 70/30); *Target Leakage* (para cada columna, AUC o R^2 de un modelo con esa sola columna por validación cruzada sobre una muestra; valores cercanos a 1 indican posible fuga, como la de los ingresos del mismo día que señala el capítulo); *Multicollinearity* (VIF recortado a 50, valores singulares de PCA tras estandarizar y coeficientes de lasso); y el *Data Quality and Insights Report*, con el poder predictivo de cada característica. **SageMaker Processing:** `SelectKBest` con `mutual_info_classif`, `chi2` o `f_classif`, ajustado solo con entrenamiento.

12. Resumen: técnica y servicio

Técnica	Data Wrangler (Canvas)	Glue DataBrew	Otra vía en AWS
Imputación, indicadora	<i>Handle missing</i>	FILL_WITH_*	Processing (SimpleImputer)
Atípicos (z , IQR, MAD)	<i>Handle outliers</i>	FLAG_OUTLIERS, REMOVE_OUTLIERS, etc.	—
Escalado lineal	<i>Process numeric</i>	SCALE	Processing
log, raíces	<i>Custom formula</i>	SKEWNESS	Processing
Box–Cox, Yeo–Johnson, cuantil	<i>Custom transform</i>	—	Processing (PowerTransformer, QuantileTransformer)
Discretización	<i>Custom formula/transform</i>	BUCKETIZATION, BINARIZATION	Processing (KBinsDiscretizer)
Ordinal	<i>Ordinal encode</i>	CATEGORICAL_MAPPING	Processing
One-hot	<i>One-hot encode</i>	ONE_HOT_ENCODING (≤ 10 niveles)	Processing
Binaria	<i>Custom formula</i> (bits)	—	Processing (category_encoders)
Hashing	<i>Vectorize/Custom transform</i>	—	Processing (FeatureHasher)
Similitud (<i>min-hash</i>)	<i>Similarity encode</i>	—	—
Target encoding	—	—	Processing, CatBoost integrado, Glue + Feature Store
WoE / IV	—	(BUCKETIZATION previa)	Processing o Glue; Linear Learner
Fecha cíclica	<i>Featurize datetime</i>	—	Processing
Rezagos, ventanas	<i>Lag/Rolling window features</i>	—	Processing
Imagen: vectores	transformaciones de imagen	—	JumpStart, Bedrock, Rekognition
CNN	—	—	—
PCA	<i>Reduce dimensionality</i>	—	PCA integrado
Tokenización, <i>stemming</i>	<i>Vectorize</i> (tokenizador)	TOKENIZATION	Processing (spaCy, NLTK)
TF-IDF, n -gramas	<i>Vectorize</i> + IDF	—	Processing (TfidfVectorizer)
Word embeddings	—	—	BlazingText, Bedrock (Titan), Comprehend
Selección	<i>análisis Quick Model, Target Leakage, Multicollinearity</i>	—	Processing (SelectKBest)

En todos los casos, los parámetros ajustados se estiman con entrenamiento (Proposición 1.3), se guardan como artefacto versionado y se aplican sin cambios en validación, prueba e inferencia; Feature Store ofrece el almacén común que evita que la definición de una característica difiera entre entrenamiento y servicio.

Referencias

- [1] G. E. P. Box y D. R. Cox (1964). An analysis of transformations. *JRSS B* 26, 211–252.
- [2] L. Breiman, J. Friedman, R. Olshen y C. Stone (1984). *Classification and Regression Trees*. Wadsworth.
- [3] A. Broder (1997). On the resemblance and containment of documents. *Compression and Complexity of Sequences*.
- [4] P. Cerda y G. Varoquaux (2022). Encoding high-cardinality string categorical variables. *IEEE TKDE* 34, 1164–1176.

- [5] D. L. Donoho y P. J. Huber (1983). The notion of breakdown point. En *A Festschrift for Erich L. Lehmann*. Wadsworth.
- [6] W. D. Fisher (1958). On grouping for maximum homogeneity. *JASA* 53, 789–798.
- [7] I. J. Good (1950). *Probability and the Weighing of Evidence*. Griffin.
- [8] T. Hastie, R. Tibshirani y J. Friedman (2009). *The Elements of Statistical Learning*, 2.^a ed. Springer.
- [9] B. Iglewicz y D. Hoaglin (1993). *How to Detect and Handle Outliers*. ASQC.
- [10] I. T. Jolliffe (2002). *Principal Component Analysis*, 2.^a ed. Springer.
- [11] O. Levy e Y. Goldberg (2014). Neural word embedding as implicit matrix factorization. *NeurIPS*.
- [12] R. J. A. Little y D. B. Rubin (2019). *Statistical Analysis with Missing Data*, 3.^a ed. Wiley.
- [13] D. Micci-Barreca (2001). A preprocessing scheme for high-cardinality categorical attributes. *SIGKDD Explorations* 3, 27–32.
- [14] T. Mikolov et al. (2013). Distributed representations of words and phrases and their compositionality. *NeurIPS*.
- [15] J. Pennington, R. Socher y C. Manning (2014). GloVe: Global vectors for word representation. *EMNLP*.
- [16] M. F. Porter (1980). An algorithm for suffix stripping. *Program* 14, 130–137.
- [17] L. Prokhorenkova et al. (2018). CatBoost: unbiased boosting with categorical features. *NeurIPS*.
- [18] D. B. Rubin (1976). Inference and missing data. *Biometrika* 63, 581–592.
- [19] R. Sennrich, B. Haddow y A. Birch (2016). Neural machine translation of rare words with subword units. *ACL*.
- [20] R. E. Shiffler (1988). Maximum Z scores and outliers. *The American Statistician* 42, 79–80.
- [21] N. Siddiqi (2006). *Credit Risk Scorecards*. Wiley.
- [22] K. Spärck Jones (1972). A statistical interpretation of term specificity. *Journal of Documentation* 28, 11–21.
- [23] J. W. Tukey (1977). *Exploratory Data Analysis*. Addison-Wesley.
- [24] K. Weinberger et al. (2009). Feature hashing for large scale multitask learning. *ICML*.
- [25] I.-K. Yeo y R. A. Johnson (2000). A new family of power transformations. *Biometrika* 87, 954–959.
- [26] J. Yosinski et al. (2014). How transferable are features in deep neural networks? *NeurIPS*.